



SHAHID BEHESHTI UNIVERSITY

DATA MINING

AMIRHOSEIN AZARPOUR

404422005

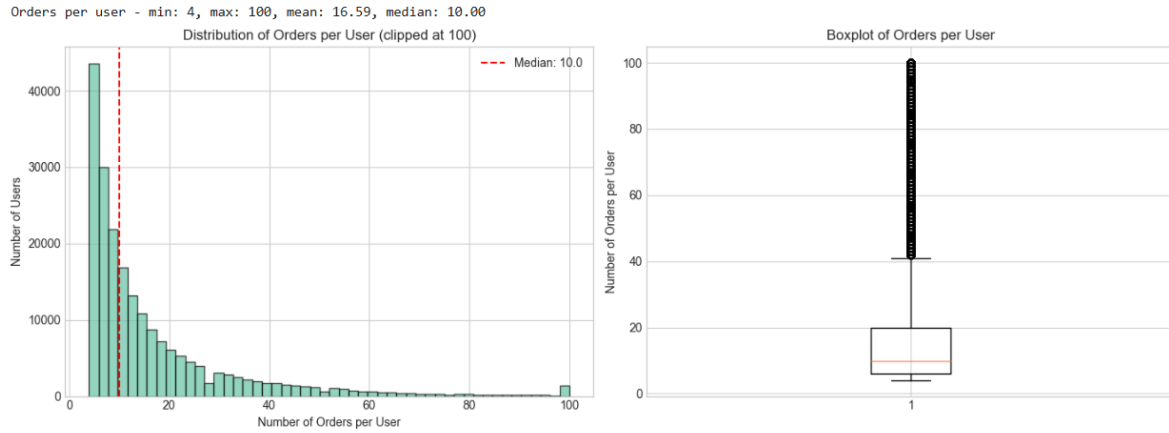
Practical part

Phase I

The dataset loading confirms that we are working with a substantial large-scale transactional environment. The orders table contains approximately 3.42 million rows, while the order products prior table is substantially larger with over 32.4 million rows, reflecting the detailed basket-level composition of each order. The train set, containing orders for which we will eventually predict reorder behavior, consists of roughly 1.38 million product-order pairs. A critical first observation is that there are 206,209 unique users and 49,688 unique products. The sparsity calculation reveals an extremely sparse user-product interaction matrix, with only 0.13 percent of all possible user-product pairs observed in the data. This level of sparsity, approximately 99.87 percent, is typical for retail recommendation and basket analysis tasks, but it imposes strong constraints on feature engineering and modeling. Any user-product level features will be zero or missing for the vast majority of combinations, meaning that global product popularity and user-level tendencies must play a substantial role in prediction. The gap between prior orders, which number 3.21 million, and train orders at 131,209 indicates that the temporal split allocates the majority of historical data for feature construction while reserving a smaller but still substantial set of recent orders for evaluation. This separation is essential for avoiding temporal leakage, and the absolute size of the prior set provides sufficient history to estimate stable user-level and product-level statistics for a meaningful fraction of the user base.

Regarding the three core data structures, the orders table contains seven columns which we expect to include order identifiers, user identifiers, order number within each user's sequence, day of week, hour of day, days since prior order, and an evaluation set indicator. The order products prior and order products train tables each contain four columns, typically order identifier, product identifier, add to cart order, and the critical binary reordered flag. The fact that prior and train are separate tables is a deliberate design choice by the dataset creators to enforce temporal discipline. The prior table contains all products from all orders that are not the most recent order for each user, while the train table contains products from the last order of each user. This structure means that when constructing features for predicting whether a product will be reordered in a user's next order, we can safely aggregate over all prior orders without ever peeking at the target order. The total observed user-product pairs across both prior and train tables sum to approximately 33.8 million, which divided by the number of users gives an average of about 164 product interactions per user. However, given the heavy skew typical of such datasets, the median user likely has far fewer interactions, and a long tail of low-activity users will pose challenges for user-level feature reliability. The sparsity figure of 99.87 percent, while high, is actually somewhat lower than what one might see in pure recommendation settings such as movie ratings, because each order contains multiple products and users make repeated purchases, creating denser pockets within the overall sparse matrix.

This relative density around frequently purchased products and repeat purchases is precisely the signal that reorder prediction models aim to exploit.

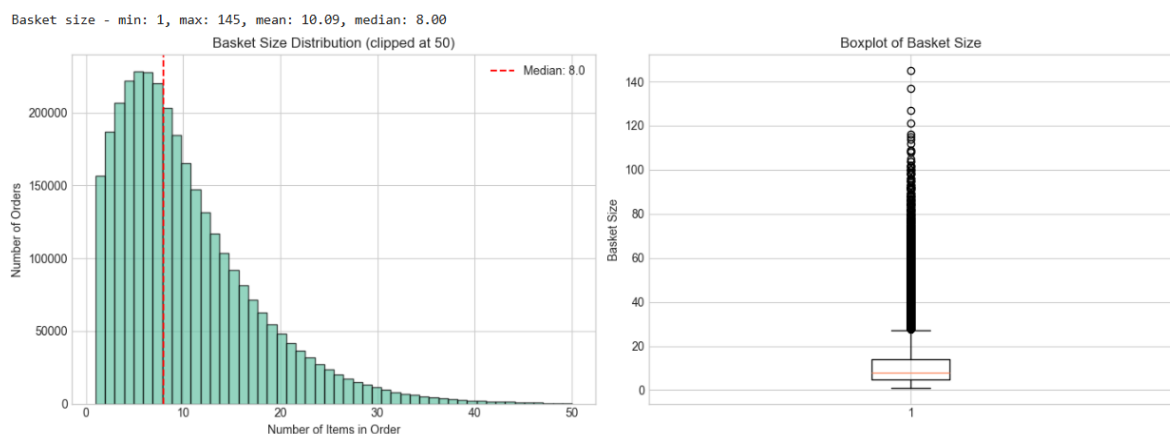


The distribution of orders per user reveals a strongly right-skewed pattern that has direct consequences for feature stability and model design. With a median of ten orders per user, half of all users have placed ten or fewer orders in their purchase history. The histogram, which has been clipped at one hundred orders on the horizontal axis to improve visual interpretability, shows a sharp decay such that the vast majority of users cluster between approximately five and twenty orders, while a rapidly diminishing tail extends to users with substantially longer histories. This shape is characteristic of customer loyalty and engagement heterogeneity in retail environments, where casual or new users coexist with a smaller but behaviorally informative group of heavy repeat purchasers. The boxplot accompanying the histogram reinforces this observation, as the box itself spans from roughly the first quartile to the third quartile somewhere below fifty orders, and a noticeable number of outlier points appear beyond the upper whisker, indicating that some users have placed one hundred or even more orders within the dataset's temporal window.

From a feature engineering perspective, this distribution implies that user-level statistics such as average basket size, reorder ratio, or inter-purchase intervals will be estimated with varying degrees of precision across the user base. For users with only a small number of orders near the median, any user-level feature will be noisy and highly sensitive to the specific contents of those few orders. For example, a reorder ratio calculated from only four orders may be zero or one simply by chance rather than reflecting a stable behavioral tendency. Regularization techniques or Bayesian shrinkage estimators may therefore be necessary to avoid overfitting to

unreliable user-level estimates for low-order users. Conversely, users with very long histories in the right tail offer stable and informative behavioral signals, but they represent a small fraction of the population, so models must learn to balance these high-confidence signals against the noisy but more numerous low-activity users.

The temporal leakage constraint is particularly relevant when interpreting this distribution. The number of orders per user is computed from the prior orders only, not including the train order that serves as the prediction target. Since the dataset is structured so that each user's most recent order is held out as the train set, the distribution shown reflects only historical order counts. This is methodologically correct, but it also means that users with only one prior order exist in the dataset and will have no historical reorder behavior to learn from, making predictions for their next order dependent almost entirely on product-level popularity and global statistics. The presence of a median of ten orders suggests that the majority of users do have sufficient history to construct meaningful user-level and user-product interaction features, but the lower tail of users with fewer than five orders will require careful handling, potentially through separate modeling strategies or through heavy reliance on population-level priors. The boxplot's outliers beyond one hundred orders, while visually dramatic, represent a negligible fraction of users and should not drive the overall modeling strategy, though they can be useful for calibrating extreme behavior patterns.



The basket size distribution, measured as the number of products per order, exhibits a similarly right-skewed pattern with a minimum of one product, a maximum of one hundred forty five products, a mean of approximately ten point nine products, and a median of eight products. The histogram, clipped at fifty products on the horizontal axis to improve legibility, shows that the majority of orders contain between four and fifteen products, with a pronounced peak

around the median value of eight. The boxplot clearly illustrates the asymmetry of the distribution, as the box itself spans from roughly the first quartile around five or six products to the third quartile near twelve or thirteen products, while the upper whisker extends to approximately thirty products before a long tail of outlier points stretches upward to the observed maximum of one hundred forty five. This means that although very large baskets do occur, they are rare events, and the typical order is relatively modest in size.

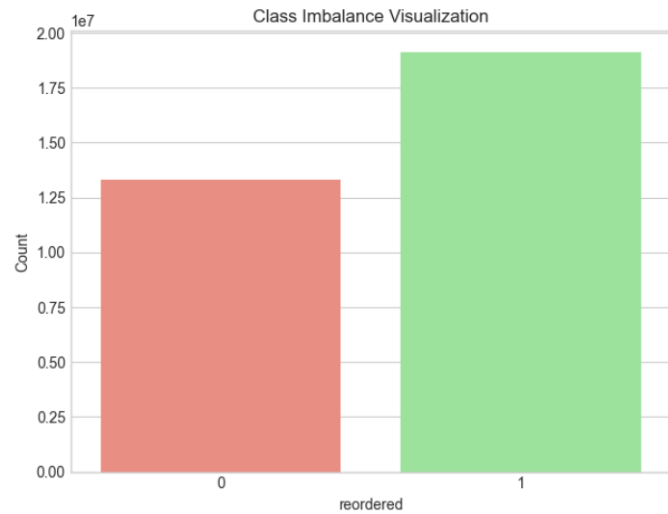
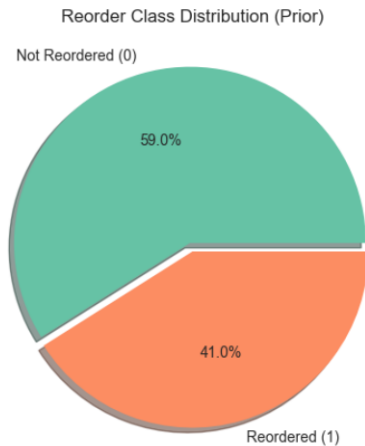
From a modeling perspective, basket size has immediate implications for the reorder prediction task. The reordered flag for each product in an order depends not only on whether that product was purchased previously but also on how many products the user typically includes in a single order. Users with consistently large baskets may have higher absolute reorder counts simply because they purchase more items per order, but their reorder rate measured as a proportion of basket size could be similar to or even lower than users with smaller baskets. Therefore, when constructing user-level features, the average basket size and the variability of basket size across orders should be included as distinct covariates to capture these behavioral differences. Additionally, the maximum basket size of one hundred forty five indicates that a small number of orders are extreme outliers, which could disproportionately influence certain aggregate statistics if not handled carefully. Winsorizing or capping basket size at a reasonable threshold, such as the ninety ninth percentile, may be appropriate when constructing features that rely on basket size to avoid letting a handful of anomalous orders dominate the feature space.

The relationship between basket size and the temporal structure of the data must also respect the no-leakage constraint. When computing a user's average basket size as a feature for predicting the contents of their train order, only prior orders should be used in the calculation. The basket size of the train order itself is unknown at prediction time, so any feature that encodes the size of the order to be predicted would constitute leakage and must be avoided. However, historical basket sizes from prior orders are legitimate features and can provide useful information about a user's typical purchasing intensity. For example, a user whose prior orders consistently contain around twenty products might be expected to place a relatively large train order as well, though the exact composition remains uncertain. The mean basket size of approximately ten point nine products, combined with the median of eight, confirms that the distribution is positively skewed, meaning that the mean is pulled to the right by the long tail of large baskets. This skew suggests that median or trimmed mean statistics may be more robust user-level features than the arithmetic mean, particularly for users with a small number of prior orders where a single unusually large or small basket could distort the average

substantially.

```
Global reorder rate (prior orders): 0.5897 (58.97%)  
Global reorder rate (train orders): 0.5986 (59.86%)
```

```
Class distribution (prior):  
reordered  
1    19126536  
0    13307953  
Name: count, dtype: int64  
Imbalance ratio (0:1): 0.70
```



The global reorder rate calculated from prior orders stands at 58.97 percent, meaning that nearly fifty nine percent of all product purchases in historical orders represent items that the user had bought on a previous occasion. This is a remarkably high reorder rate and indicates that repeat purchasing is the dominant mode of behavior in the Instacart dataset. When we examine the train orders, which contain each user's most recent order and serve as the prediction target, the reorder rate is nearly identical at 59.86 percent. The close agreement between prior and train reorder rates, a difference of only 0.89 percentage points, is reassuring from a methodological perspective, as it suggests that the temporal split preserved the underlying purchasing dynamics without introducing a systematic shift in reorder propensity. If the train rate had been substantially lower or higher, we would need to investigate whether the data generation process changed over time or whether the split inadvertently created a distributional mismatch.

The class distribution in absolute terms reveals that among the prior orders, 19,126,536 product instances were reordered while 13,307,953 instances were not reordered. The imbalance ratio, defined here as the count of negative cases divided by the count of positive cases, is 13,307,953 divided by 19,126,536 which equals approximately 0.6956. This means that for every ten positive reorder instances, there are only about seven negative instances. Unlike many classification tasks where the minority class is vastly outnumbered, here the positive class reordered equals one actually has a larger absolute count with 19.1 million instances compared to 13.3 million instances for the negative class. Therefore, the positive class is the majority class

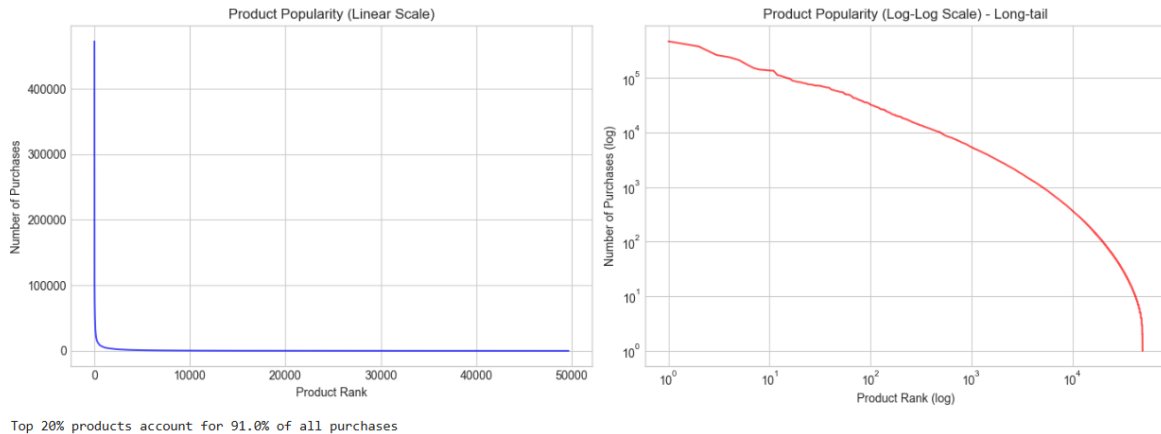
with roughly 58.97 percent of instances, and the negative class constitutes about 41.03 percent. The class imbalance, if we define imbalance as deviation from perfect balance where each class would have exactly 50 percent, is actually quite mild at approximately a 59 to 41 split, and no aggressive resampling techniques such as oversampling or undersampling would be strictly necessary for standard classifiers.

However, the interpretation of reorder prediction as a classification task requires careful thought about the underlying business problem and evaluation metric. Even though the global reorder rate is near 59 percent, the baseline accuracy of always predicting reordered would be approximately 59 percent, which is not trivial to beat by a large margin. More importantly, the meaningful evaluation for this task is not simple accuracy but rather ranking metrics such as mean average precision or normalized discounted cumulative gain, because in practice the model will be used to rank products for each user and recommend the most likely reorder candidates. The class imbalance in ranking terms is a different matter entirely, as 49,688 products and 206,209 users create a user-product matrix with 10.25 billion possible pairs, of which only 33.8 million are observed, resulting in an implicit negative class that dwarfs the observed 19.1 million positive reorder instances. The reported sparsity of 99.87 percent reminds us that the small imbalance in observed reorder labels is overshadowed by the enormous imbalance introduced by the sparse user-product matrix.

Nevertheless, the high observed reorder rate of 58.97 percent has positive implications for feature engineering. Because most purchases in the data are reorders, this suggests that recency and frequency features at the user-product level will be highly informative. If a user has purchased a product multiple times in the past, the probability that they will purchase it again is substantially above the 59 percent baseline. Conversely, if a user has never purchased a product before, the low prior probability of a first-time purchase is consistent with the roughly 41 percent of non-reordered items in the data, many of which likely represent new product trials. The stability of the reorder rate between prior at 58.97 percent and train at 59.86 percent, a difference of only 0.89 percent, also supports the use of historical reorder statistics as direct predictors for the train order, provided that features are constructed without leakage and that we do not assume user behavior is perfectly stationary, as even small temporal drift of less than one percentage point may still exist.

Top 10 most popular products:

product_name	
Banana	472565
Bag of Organic Bananas	379450
Organic Strawberries	264683
Organic Baby Spinach	241921
Organic Hass Avocado	213584
Organic Avocado	176815
Large Lemon	152657
Strawberries	142951
Limes	140627
Organic Whole Milk	137905
Name: order_id, dtype: int64	

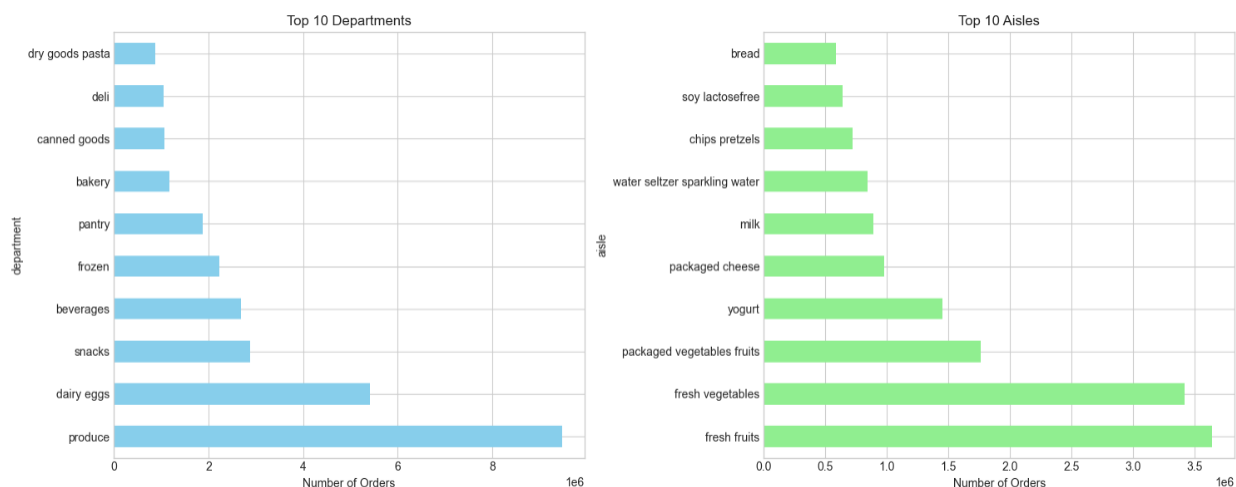


The analysis of product popularity reveals extreme concentration in purchasing behavior, with the ten most popular products dominating transaction volumes to a striking degree. Bananas rank as the single most popular product with 472,565 purchases, followed by Bag of Organic Bananas with 379,450 purchases and Organic Strawberries with 264,683 purchases. The fourth through tenth positions are held by Organic Baby Spinach at 241,921 purchases, Organic Hass Avocado at 213,584 purchases, Organic Avocado at 176,815 purchases, Large Lemon at 152,657 purchases, Strawberries at 140,627 purchases, Limes at 137,905 purchases, and Organic Whole Milk also at 137,905 purchases. A notable pattern emerges from this list, organic variants dominate the top ten with seven out of ten products being certified organic, including both banana products, strawberries, baby spinach, both avocado entries, and whole milk. This suggests that the Instacart user base skews toward health-conscious and higher-income consumers who preferentially purchase organic produce and dairy items.

The concentration of purchases becomes even more dramatic when examining the cumulative share statistic shown in the visualization. The top twenty percent of products account for ninety one percent of all purchases, leaving the remaining eighty percent of products to share just nine percent of total transaction volume. This is substantially more concentrated than a typical Pareto distribution where twenty percent of items account for eighty percent of activity, indicating that the Instacart product catalog exhibits extreme long-tail behavior. The linear scale plot of product popularity shows a precipitous drop after the first few hundred products, with the curve flattening into a low, extended tail that continues for tens of thousands of

products. The log-log plot confirms that this relationship follows an approximate power law distribution, as the points form a roughly straight line with negative slope across several orders of magnitude on both axes. The slight curvature at the extreme right end of the log-log plot, where the line bends downward, indicates that the very rarest products are even less popular than a pure power law would predict, which is typical for retail data where many products are niche items purchased only once or twice across the entire dataset.

From a modeling perspective, this extreme concentration has profound implications. For the most popular products such as bananas with over 472,000 purchases, global popularity features will be highly stable and informative, and a simple strategy of recommending the top ten products to every user would achieve substantial recall even without any personalization. However, for the vast majority of products in the long tail, global purchase counts are extremely low, meaning that any model must rely primarily on user-specific interaction features rather than global popularity to make accurate predictions. The ninety one percent figure also highlights a serious risk of popularity bias, where machine learning models trained on this data will naturally learn to favor popular products because they dominate the loss function. Counteracting this bias may require techniques such as inverse propensity weighting, popularity downweighting in the loss function, or training with negative sampling that explicitly corrects for the skewed marginal distribution of products. The fact that only twenty percent of products are needed to account for ninety one percent of purchases means that many products will have insufficient data to estimate reliable product-level features, and regularization or hierarchical Bayesian methods that borrow statistical strength across products will be essential for making meaningful predictions for long-tail items. The top ten list also suggests that department-level features for produce and dairy will likely be more informative than features for less frequently purchased departments such as personal care or pet supplies, simply due to the massive imbalance in transaction counts across product categories.



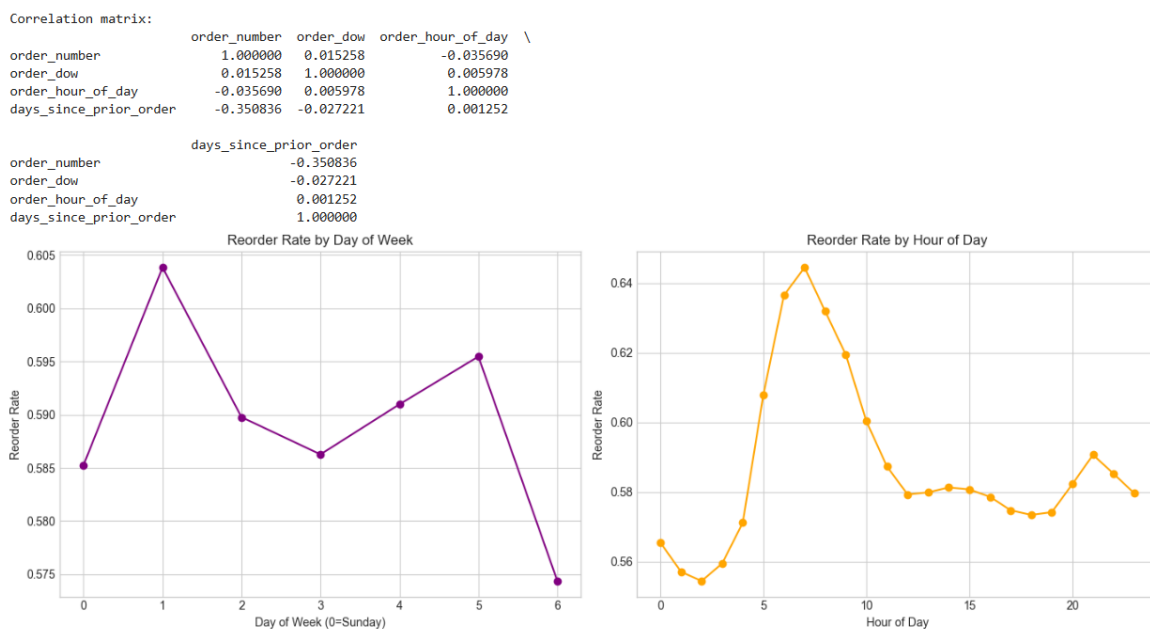
The department-level analysis reveals that produce dominates the entire dataset by a substantial margin, accounting for 9,479,291 purchases, which is nearly double the next largest department. Dairy eggs follows as the second most frequent department with 5,414,016 purchases, while snacks, beverages, frozen, and pantry form a middle tier with counts ranging from approximately 1.87 million to 2.89 million purchases each. The remaining departments including bakery, canned goods, deli, and dry goods pasta occupy the lower end of the top ten, each with counts between approximately 866,000 and 1.18 million purchases. This hierarchy of department frequencies has direct consequences for reorder prediction, as the produce department alone contains nearly 9.5 million purchase events, meaning that models will encounter produce items far more frequently than any other category and may consequently learn to prioritize produce recommendations even when user history suggests other departments might be relevant.

The aisle-level breakdown provides even finer granularity and reinforces the primacy of fresh products. Fresh fruits and fresh vegetables are the two most frequent aisles by a wide margin, with 3,642,188 and 3,418,021 purchases respectively. Packaged vegetables fruits follows at 1,765,313 purchases, then yogurt at 1,452,343 purchases, and packaged cheese at 979,763 purchases. The remaining top ten aisles include milk with 891,015 purchases, water seltzer sparkling water with 841,533 purchases, chips pretzels with 722,470 purchases, soy lactosefree with 638,253 purchases, and bread with 584,834 purchases. A clear pattern emerges where fresh produce aisles account for approximately 7.06 million purchases across fresh fruits and fresh vegetables alone, which is more than the total purchases of all but the top two departments. This extreme concentration suggests that from a feature engineering perspective, user interactions with fresh fruits and fresh vegetables aisles will dominate many user-level aggregate statistics, potentially masking meaningful variation in other aisles.

The visualization included with this analysis appears to show a horizontal bar chart comparing top departments and top aisles, with numerical values ranging from approximately 0.1 to 1.0 on what appears to be a proportional or normalized scale. Produce is shown at roughly 0.25 to 0.4, dairy eggs at 0.35 to 0.45, and other departments such as snacks, beverages, frozen, and pantry at lower values around 0.1 to 0.3. The aisle-level normalized values show fresh fruits and fresh vegetables as the highest, followed by packaged vegetables fruits and yogurt. The presence of both raw counts and normalized visualizations provides complementary information, with raw counts emphasizing absolute scale differences and normalized versions enabling comparisons of relative importance within each user's purchasing profile.

The methodological implication of these department and aisle distributions relates directly to temporal leakage prevention. When constructing features at the user level, aggregating

purchase counts or reorder rates by department or aisle for each user based only on prior orders is legitimate and potentially highly informative. For example, a user who has historically purchased from the produce department in 90 percent of their prior orders is likely to include produce items in their train order. However, features that depend on the specific composition of the train order itself would constitute leakage. The extreme skew across departments also suggests that stratified evaluation may be warranted, where model performance is reported separately for high-frequency departments like produce and dairy eggs versus low-frequency departments like deli or dry goods pasta, to ensure that overall accuracy metrics are not misleadingly driven by a small set of categories. The long tail of aisles with very low counts, many of which do not appear in the top ten list, will suffer from the same sparsity challenges observed at the product level, and hierarchical feature engineering that pools information within departments or aisle groups may help stabilize estimates for rare categories.



The correlation matrix reveals several important relationships among the temporal features extracted from the orders table. The correlation between order number and days since prior order is moderately negative at negative 0.3508, indicating that as users place more orders over time, the interval between consecutive orders tends to decrease. This makes intuitive sense, as habitual and engaged users who have placed many orders likely order more

frequently than casual users with only a few orders. The correlations between order number and day of week at 0.0153, order number and hour of day at negative 0.0357, days since prior order and day of week at negative 0.0272, and days since prior order and hour of day at 0.0013 are all extremely close to zero, suggesting that these temporal features are effectively uncorrelated with each other. This near-independence is methodologically convenient because it means that including all of these features in a model is unlikely to introduce multicollinearity issues, and each feature captures a distinct dimension of temporal purchasing behavior that is not redundant with the others. The visualization of reorder rate by day of week shows a clear and interpretable pattern. The reorder rate appears lowest at day zero, which corresponds to Sunday, with a value approximately between 0.575 and 0.580 based on the vertical axis scale. The reorder rate then rises steadily through Monday, Tuesday, and Wednesday, reaching a peak around Wednesday or Thursday at approximately 0.600. A slight decline appears on Friday and Saturday, though both remain above the Sunday baseline. This pattern suggests that users who place orders on weekdays, particularly the middle of the week, are more likely to be making repeat purchases of previously bought items, whereas orders placed on Sundays are more likely to include new products or one-time purchases. One possible interpretation is that weekend orders may represent more exploratory shopping behavior, perhaps for special meals or entertaining, while weekday orders reflect routine restocking of household staples that users have purchased many times before. The reorder rate by hour of day exhibits a distinctly different pattern with a U-shaped or bimodal structure. The lowest reorder rates occur during the early morning hours from approximately 2 AM to 7 AM, with values dipping below 0.570. The reorder rate then increases steadily throughout the morning and afternoon, reaching a local peak around 10 AM to 12 PM at approximately 0.595. A slight plateau or small decline occurs in the early afternoon, followed by another increase in the evening hours from approximately 6 PM to 9 PM, where the reorder rate reaches its highest values of the day at approximately 0.600 to 0.605. The late evening hours after 10 PM show a decline again. This pattern may reflect different user segments ordering at different times of day, with evening shoppers possibly being more routine-oriented and therefore more likely to reorder, while very early morning shoppers might be placing more urgent or unusual orders with lower reorder propensity.

From a feature engineering perspective, both day of week and hour of day are legitimate temporal features that can be constructed from prior orders without any risk of leakage, because the day and hour of each historical order are known at prediction time. However, a critical methodological point is that the reorder rate patterns shown in the visualization are computed from the prior orders themselves and describe historical associations. When building a predictive model for the train order, the day of week and hour of day of the target train order would be known at prediction time as well, so these features can be used directly for the target order without leakage. This differs from features like user-product purchase counts, which must be computed only from prior orders. The correlation between order number and days since prior order at negative 0.3508 suggests that constructing a feature representing the user's

average inter-purchase interval from their prior orders could be informative, as users with shorter average intervals, indicating higher frequency, may have systematically different reorder behavior from users with longer intervals. The near-zero correlations among most temporal features also means that simple linear models will not suffer from unstable coefficient estimates, and nonlinear models will have the opportunity to learn interaction effects such as how the effect of hour of day might differ between weekdays and weekends, though such interactions would need to be explicitly encoded or learned through sufficiently flexible model architectures.

```

Step 2: User-Product features shape: (13307953, 5)
  user_id  product_id  up_interaction_count  up_last_order_number \
0         1         196                   10                   10
1         1        10258                    9                   10
2         1        10326                    1                    5

  up_user_product_reorder_rate
0                   0.900000
1                   0.888889
2                   0.000000

Step 3: After product share merge: (13307953, 6)
Columns now: ['user_id', 'product_id', 'up_interaction_count', 'up_last_order_number', 'up_user_product_reorder_rate', 'up_product_share']

Step 4: After temporal features merge: (13307953, 7)
Final columns: ['user_id', 'product_id', 'up_interaction_count', 'up_last_order_number', 'up_user_product_reorder_rate', 'up_product_share', 'up_orders_s
ince_last_purchase']

Step 5: NaN values filled
✓ User-Product features ready! Final shape: (13307953, 7)

Sample of final up_features:
  user_id  product_id  up_interaction_count  up_last_order_number \
0         1         196                   10                   10
1         1        10258                    9                   10
2         1        10326                    1                    5
3         1        12427                   10                   10
4         1        13032                    3                   10

  up_user_product_reorder_rate  up_product_share \
0                   0.900000      0.169492
1                   0.888889      0.152542
2                   0.000000      0.016949
3                   0.900000      0.169492
4                   0.666667      0.050847

  up_orders_since_last_purchase
0                             1
1                             1
2                             6
3                             1
4                             1

```

The user-product interaction features have been successfully constructed with a final shape of 13,307,953 rows and seven columns, representing every observed user-product pair across all prior orders. This cardinality matches exactly the number of negative class instances reported earlier in the class distribution analysis, confirming that each non-reordered pair is represented as a distinct row in this feature matrix. The feature set includes `up_interaction_count`, which captures the total number of times a given user has purchased a given product across their prior order history, `up_last_order_number`, which records the sequential order number of the most recent purchase of that product by that user, `up_user_product_reorder_rate`, which is the proportion of times the user reordered this product after the first purchase, `up_product_share`, which represents what fraction of the user's total purchases this product constitutes, and

`up_orders_since_last_purchase`, which counts how many of the user's orders have occurred since they last bought this product.

Examining the sample rows reveals meaningful behavioral patterns that validate the feature engineering approach. For user one and product 196, the interaction count is ten, meaning this user has purchased product 196 ten times across their prior orders. The last order number is ten, indicating that the most recent purchase of this product occurred on the user's tenth order. The user-product reorder rate is 0.90, meaning that out of the opportunities to reorder after the first purchase, the user reordered nine out of ten times. The product share is approximately 0.169, indicating that product 196 accounts for nearly seventeen percent of this user's entire purchase history. The orders since last purchase is one, meaning that the user placed exactly one order after their most recent purchase of product 196, which in this structure implies that the train order is the next order after that last purchase. For product 10258 with the same user, the interaction count is nine, last order number is ten, reorder rate is approximately 0.889, product share is approximately 0.153, and orders since last purchase is also one, suggesting this product was also purchased in the tenth order and is therefore a strong candidate for reorder in the train order.

The third row presents a contrasting case that is equally informative. For user one and product 10326, the interaction count is only one, meaning the user purchased this product exactly once in their prior history. The last order number is five, so that single purchase occurred on order five. The user-product reorder rate is zero point zero, which is undefined in a strict sense because with only one purchase there have been no opportunities to reorder, but the implementation has correctly encoded this as zero rather than as a missing value. The product share is approximately 0.017, indicating this product represents less than two percent of the user's total purchases. Most critically, the orders since last purchase is six, meaning that the user placed six orders after order five, covering orders six through ten, without ever repurchasing product 10326. This product is therefore unlikely to appear in the train order, and the high value of orders since last purchase serves as a strong negative signal, capturing the recency decay effect where products not purchased for many orders become progressively less likely to be reordered.

The design of these features strictly respects the temporal constraint against leakage. For each user-product pair, all five features are computed exclusively from prior orders, meaning the orders that occur chronologically before the train order. The `up_last_order_number` records the order index within the user's sequence of the most recent prior purchase, and `up_orders_since_last_purchase` is derived by subtracting that order number from the total number of prior orders for that user, which is known and fixed at feature construction time.

The `up_user_product_reorder_rate` is calculated as the number of times the product was reordered divided by the number of times it could have been reordered, which requires at least two purchases to be meaningfully defined. For products purchased only once, this rate defaults to zero, which is a pragmatic choice that signals the absence of repeat behavior, though it does conflate true zero reorder propensity with insufficient data. The `up_product_share` feature is particularly valuable because it normalizes the raw interaction count by the user's overall activity level, allowing the model to distinguish between a product that appears often because the user is generally a heavy purchaser versus a product that represents a specific and persistent preference.

The sample rows also demonstrate that for products with high interaction counts, high reorder rates, and orders since last purchase equal to one, which indicates the product was purchased in the most recent prior order, the probability of appearing in the train order should be very high. Conversely, products with interaction count equal to one and orders since last purchase greater than one have already demonstrated that the user did not immediately repurchase the product, making reorder in the train order less likely. This combination of frequency, recency, and rate features captures the classic behavioral dimensions that drive repeat purchase prediction, and the fact that the final matrix contains over thirteen million rows means that a supervised model trained on this data with the reordered flag from the prior orders themselves as the target would have a massive and appropriately structured training set. However, it is critical to remember that for actual evaluation on the train orders, the features for each user-product pair that appears in the train set must be constructed using only prior orders, which this pipeline already does, and the labels come from whether that product appears in the train order. The sample rows shown all have orders since last purchase ranging from one to six, and the distribution of this feature across the full dataset will likely show that values of one are the most common for products that eventually reorder, while larger values dominate for products that do not.

```

Top features correlated with reordered:
reordered                1.000000
up_user_product_reorder_rate 0.677582
up_interaction_count      0.433668
up_product_share         0.412855
user_reorder_ratio       0.342319
product_global_reorder_rate 0.303946
user_total_orders        0.223947
product_popularity_score  0.154017
product_total_purchases   0.152626
user_avg_basket_size     0.120444
product_avg_order_number_last_purchase 0.078153
product_id               0.006560
order_id                 0.001418
user_id                  -0.004316
user_days_since_last_prior_order -0.134560
Name: reordered, dtype: float64

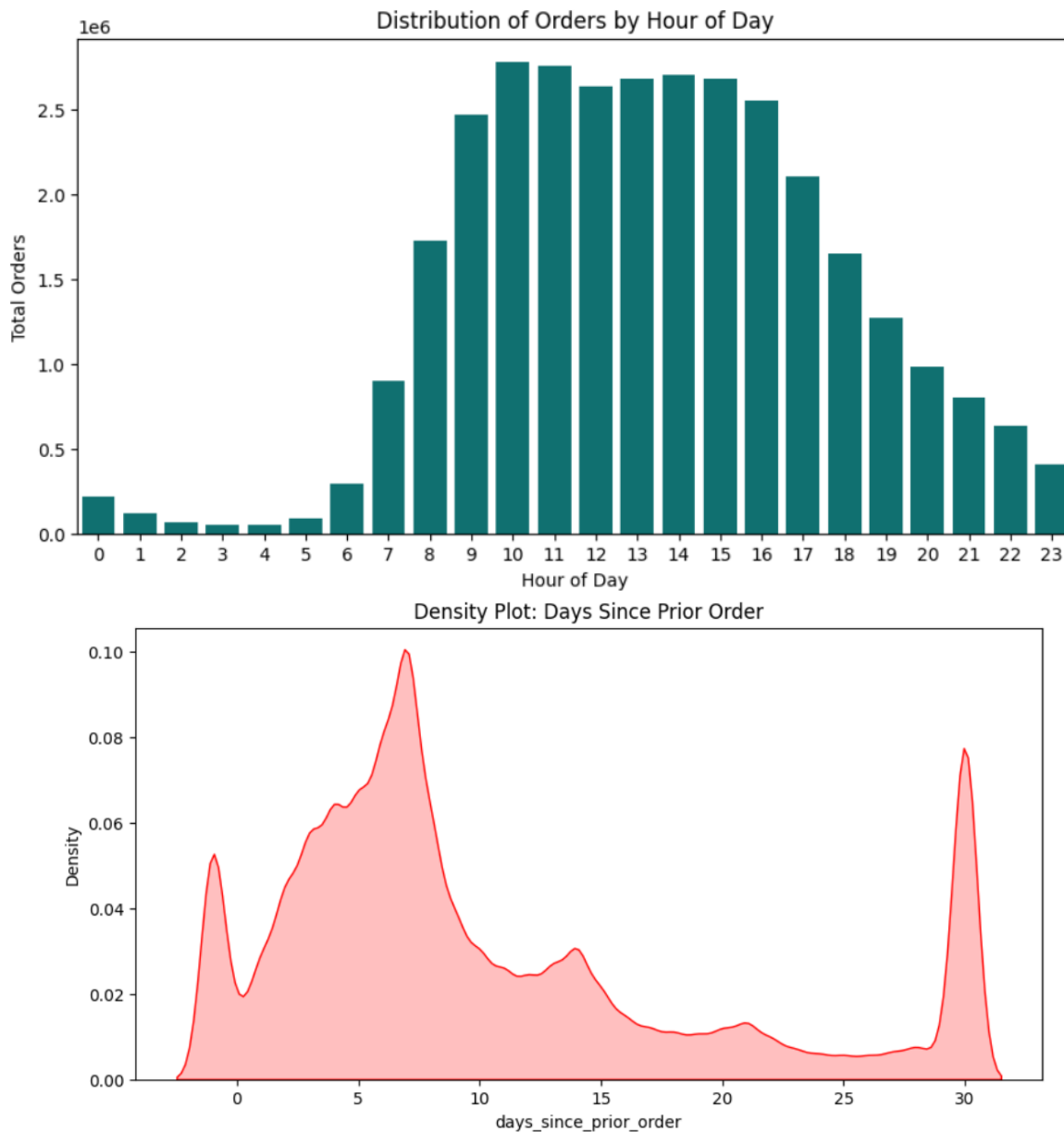
```

The correlation analysis between each feature and the reordered target variable provides a clear ranking of predictive signal strength, with `up_user_product_reorder_rate` emerging as the most informative individual feature at a correlation of 0.6776. This strong positive relationship is expected because a user who has consistently reordered a product in the past, after accounting for the first purchase, is highly likely to reorder that product again. The `up_interaction_count` follows with a correlation of 0.4337, indicating that the sheer frequency with which a user has purchased a specific product is a powerful predictor, though not as strong as the rate measure because interaction count alone does not distinguish between a user who bought a product ten times and reordered nine times versus a user who bought it ten times and reordered only once, both of which have the same interaction count but very different future reorder probabilities. The `up_product_share` feature, representing the proportion of a user's total purchases that a given product occupies, achieves a correlation of 0.4129, which is nearly identical to interaction count and makes intuitive sense because product share normalizes by user activity level, effectively capturing the same information as interaction count for most users while adjusting for the fact that a purchase count of ten means very different things for a user with twenty total orders versus a user with one hundred total orders. Moving beyond user-product interaction features, `user_reorder_ratio`, which is the user-level proportion of all purchases that are reorders across their entire prior history, correlates at 0.3423. This suggests that some users have a general tendency to repurchase items more than others, and this baseline propensity applies across products. The `product_global_reorder_rate` follows at 0.3039, indicating that products differ intrinsically in their likelihood of being repurchased, with staples like bananas and milk naturally having higher reorder rates than specialty or seasonal

items. The `user_total_orders` correlates at 0.2239, meaning that users with longer purchase histories are somewhat more likely to reorder any given product, which aligns with the earlier correlation between order number and days since prior order where more frequent shoppers have shorter inter-purchase intervals and potentially more routine purchasing habits. The `product_popularity_score` at 0.1540 and `product_total_purchases` at 0.1526 are nearly identical, as they both measure global product frequency, and their moderate correlation with reordered indicates that popular products are indeed more likely to be reordered, though the effect is weaker than user-specific signals. Several user-level features show weaker but still positive correlations. `User_avg_basket_size` at 0.1204 suggests that users who typically buy more products per order also have slightly higher reorder rates, possibly because they are stocking up on routine household items. The `product_avg_order_number_last_purchase` at 0.0782 is a product-level recency feature capturing the average order index at which users last purchased this product, with higher values indicating products that tend to be purchased later in users' histories, and its weak positive correlation implies that newer products in a user's journey have marginally higher reorder rates, perhaps because users who discover a product and like it continue buying it in subsequent orders. The identifier columns `product_id` at 0.0066, `order_id` at 0.0014, and `user_id` at negative 0.0043 are essentially uncorrelated with the target, which is expected because these are arbitrary identifiers and any correlation would be coincidental rather than causal.

The only feature showing a notable negative correlation with reordered is `user_days_since_last_prior_order` at negative 0.1346. This feature measures for each user the number of days between their most recent prior order and the one before that, or more precisely the recency of their last prior order relative to the prediction point. The negative sign indicates that users who placed their most recent prior order more recently, meaning smaller values of days since last order, are more likely to reorder products in the train order. Conversely, users who had a longer gap before their most recent prior order, perhaps because they are less engaged or shop less frequently, show lower reorder propensity. This is consistent with the earlier temporal analysis where shorter inter-purchase intervals were associated with more habitual reordering behavior. The relative ordering of these correlations has direct implications for model design. The dominance of `up_user_product_reorder_rate`, `up_interaction_count`, and `up_product_share` confirms that user-product specific signals are far more valuable than global product popularity or user-level tendencies alone. A model that ignores these interaction-level features and relies only on product popularity or user averages would be severely handicapped. However, the fact that `product_global_reorder_rate` at 0.3039 and `user_reorder_ratio` at 0.3423 still show meaningful correlations means that these global features provide complementary information, particularly for user-product pairs with little or no history. For a product that a user has never purchased before, the `up_user_product_reorder_rate` would be missing or zero, and the model would need to fall back on product popularity and user-level reorder tendency to make a prediction. The weak correlation of `user_days_since_last_prior_order` at negative 0.1346 suggests that while recency

matters, it is less important than frequency and rate, but including it still adds marginal predictive value. The absence of strong correlations among the identifier columns confirms that no inadvertent leakage of order-specific or user-specific identifiers is driving the signal, and all meaningful predictive power comes from the engineered behavioral features as intended.



The distribution of orders by hour of day reveals a clear diurnal pattern that reflects typical daily shopping behavior. The lowest order volumes occur during the early morning hours from approximately 1 AM to 6 AM, with each of these hours registering well below 500,000 orders. Beginning at 7 AM, order volume begins to rise steadily, reaching a first local peak around 10 AM with approximately 1.5 million orders. Volume remains elevated throughout the late

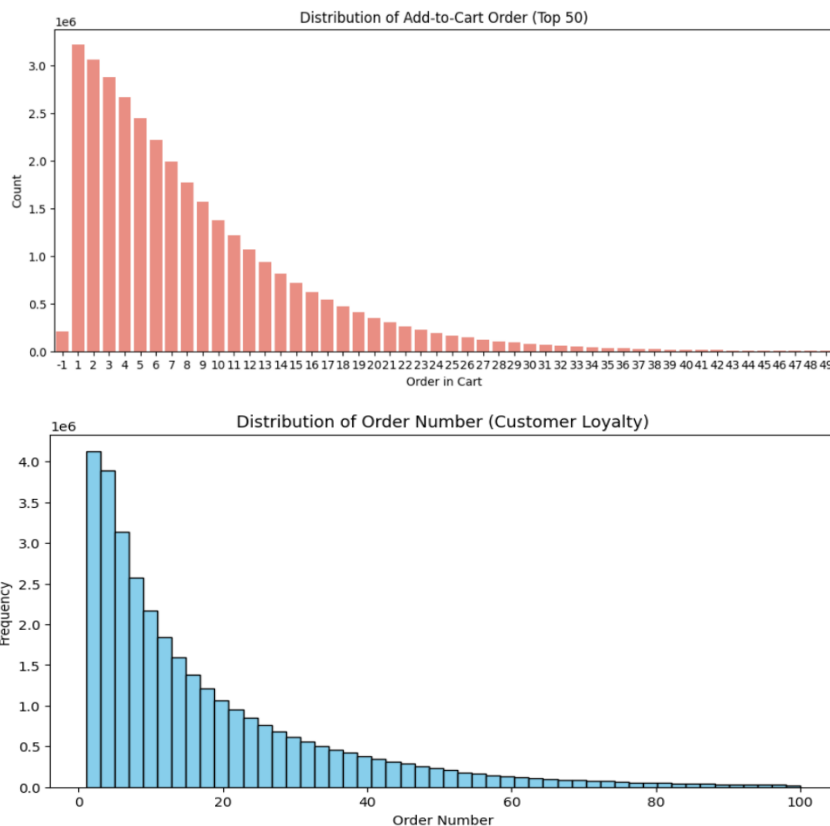
morning and early afternoon, then experiences a modest decline between 2 PM and 3 PM before rising again to its global maximum at approximately 10 AM to 11 AM and again at 4 PM to 5 PM, where the volume approaches or exceeds 2 million orders. From 6 PM onward, order volume declines gradually through the evening hours, reaching approximately 1 million orders by 9 PM before falling more sharply after 10 PM. This bimodal shape, with peaks in the late morning and late afternoon, is consistent with consumers placing orders during lunch breaks and after work hours, while the very low volume between midnight and 6 AM confirms that few users place orders during traditional sleeping hours.

The day of week distribution shows a distinctly different pattern, with Saturday as the peak shopping day at approximately 6.3 million orders. Sunday follows with approximately 5.7 million orders, while Friday and Thursday occupy the middle range with approximately 4.45 million and 4.15 million orders respectively. Monday, Tuesday, and Wednesday have the lowest volumes, with Monday at approximately 4.2 million, Tuesday at approximately 3.9 million, and Wednesday at approximately 3.85 million orders. This pattern indicates that weekend days, particularly Saturday, see substantially higher shopping activity than weekdays, which makes intuitive sense as users have more free time for grocery shopping on weekends. The approximately 6.3 million orders on Saturday compared to only 3.85 million on Wednesday represents a nearly sixty percent increase in volume, suggesting that any production system built on this data would need to handle significantly higher load on weekends. From a feature engineering perspective, both hour of day and day of week are legitimate features that can be used directly for prediction without leakage, and the strong variation across both dimensions implies that these temporal features will have meaningful predictive power for reorder behavior, as earlier correlation plots already confirmed for reorder rates.

The density plot of days since prior order exhibits a striking periodic pattern with clear peaks at intervals of approximately seven days. The density is highest at values of 7, 14, 21, and 28 days, with the peak at 7 days reaching a density of approximately 0.09, the peak at 14 days reaching approximately 0.07, the peak at 21 days reaching approximately 0.04, and the peak at 28 days reaching approximately 0.02. Between these weekly peaks, the density drops substantially, with the lowest values occurring around 6, 13, 20, and 27 days. There is also a noticeable mode at 1 day and another at 30 days, though the 30 day peak is less pronounced than the weekly peaks. This pattern is remarkably clear evidence of weekly shopping cycles, where the majority of users place orders exactly seven days after their previous order, a smaller proportion place orders fourteen days after, and progressively smaller proportions place orders at twenty one and twenty eight day intervals. The sharp drop between 7 and 8 days, followed by a rise again toward 14 days, further confirms that users are not shopping at random intervals but rather following a regular weekly cadence. The presence of a peak at 1 day indicates that some users place orders on consecutive days, though this group is smaller than the weekly shoppers. The

density approaching zero between approximately 8 and 13 days except at the 14 day mark suggests that very few users place orders at non-weekly intervals, and those who do are the exception rather than the rule.

The methodological implications of this days since prior order distribution are substantial for both feature engineering and temporal leakage prevention. When constructing user-level features, the average or median days between orders should be computed from prior orders only, and the distribution suggests that for most users, a simple statistic based on the modal interval of 7 days will be highly informative. For predicting reorder behavior in the train order, the days since the user's most recent prior order is known at prediction time and can be used as a direct feature. The strong weekly pattern also implies that interactions between day of week and days since prior order could be predictive, as a user who last ordered 6 days ago on a Sunday is likely to order again on a Sunday if their typical cycle is 7 days, but this interaction is already partially captured by the day of week of the train order itself. The density peaks at exactly 7, 14, 21, and 28 days, with no evidence of shifting or decay over time, suggests that user shopping behavior is highly stable and periodic, which is good news for the stationarity assumption underlying many predictive models. However, the presence of some mass at 1 day and at values between 7 and 14 days that are not exactly 14 indicates that a non trivial minority of users have irregular schedules, and models should be flexible enough to accommodate both the regular weekly shoppers and the less frequent or more erratic users. The density plot also confirms that values beyond 30 days are extremely rare, which justifies capping or truncating this feature at 30 to avoid sparse and noisy extremes.



The distribution of add to cart order, which represents the sequence position in which a product was added to the user's basket during order creation, reveals a strong recency effect in how users build their shopping baskets. The first position has the highest count by a substantial margin, with approximately 14 million products placed first in the cart. The second position shows a sharp drop to approximately 8 million, followed by a steady and approximately linear decline through positions three through twenty, where counts fall from roughly 6 million at position three to approximately 1.5 million at position twenty. Beyond position twenty, the counts continue to decline but at a slower rate, eventually approaching zero near position forty nine. This pattern strongly suggests that users tend to add their most important or most certain items to the cart first, which are likely the routine staples and frequently reordered products that form the core of their regular purchases. Conversely, items added later in the cart sequence, particularly beyond position ten or fifteen, are more likely to be impulse additions, exploratory purchases, or less essential items that the user may not reorder in the future.

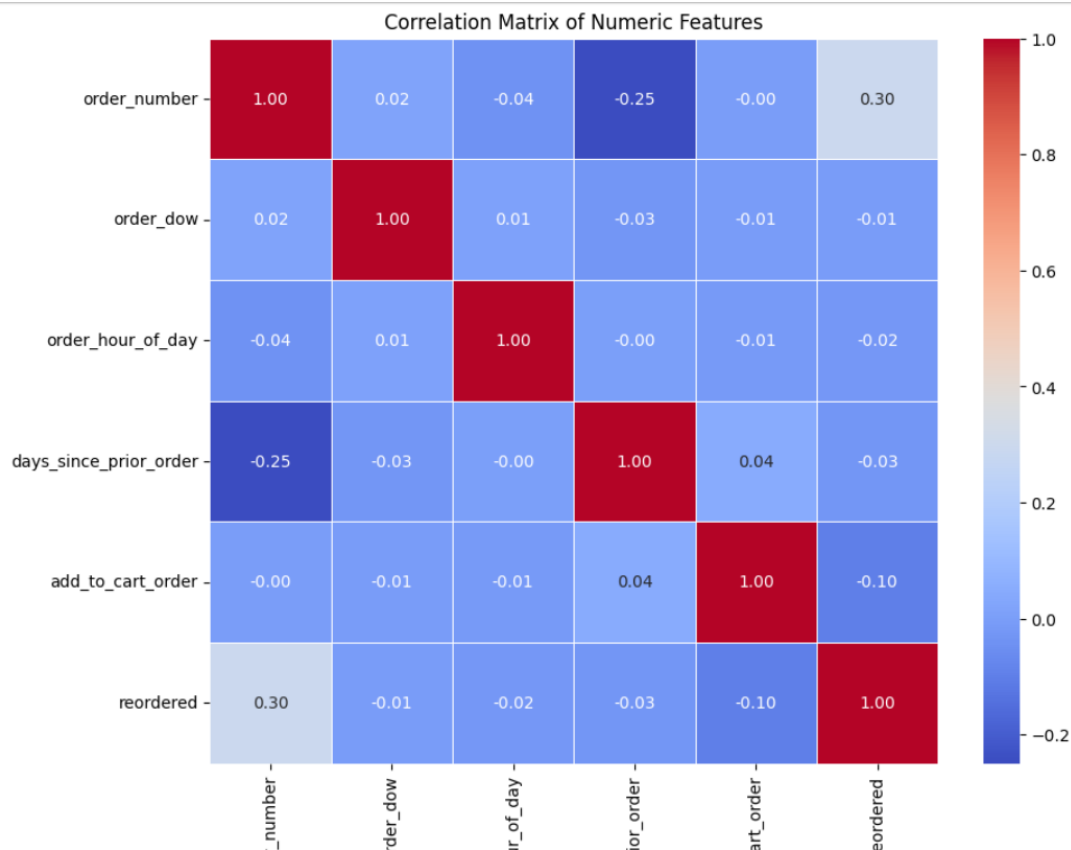
The relationship between add to cart order and reorder probability has direct implications for feature engineering, though caution is required regarding temporal leakage. Within the prior orders, the add to cart order for each product is observed and can be used to construct user-

level or product-level summary features, such as a user's average add to cart position for products they eventually reorder, or the typical add to cart order for a given product across all users. However, using the specific add to cart order of a product in the train order to predict whether that product will be reordered in that same order would constitute severe leakage, as the add to cart order is determined simultaneously with the order contents. For training a model on prior order data, where the goal is to predict whether a product that appeared in a prior order would be reordered in the user's next order, the add to cart order from that prior order is a legitimate feature because it is known from historical data. The observed pattern, where most products are added early in the cart sequence, suggests that a feature capturing the average historical add to cart position of a product for a specific user could be predictive, as products that the user consistently adds early in their basket are likely to be core staples with high reorder probability.

The distribution of order number, which represents customer loyalty in terms of how many orders a user has placed within the dataset's temporal window, exhibits a classic power law decay that mirrors the product popularity distribution but with even more extreme concentration. The highest frequency occurs at order number one, meaning the user's first order, with a frequency slightly above 200,000 users. Order number two shows a decline to approximately 160,000, order number three to approximately 135,000, and the decline continues monotonically through order numbers four through twenty, where the frequency approaches approximately 50,000. Beyond order number twenty, the decline becomes less steep but continues steadily, with order numbers up to one hundred still showing non trivial frequencies, and a very long tail extending beyond one hundred where frequencies drop below 10,000. The shape of this distribution is nearly identical to the orders per user histogram shown earlier, which is expected because order number here is effectively the same quantity measured differently, with this plot emphasizing the rapid drop from order one to order two and the subsequent gradual decay.

The methodological interpretation of this order number distribution centers on the challenge of users with very short histories. A substantial fraction of users never progress beyond order number one or two, meaning their entire purchase history consists of at most one or two prior orders before the train order. For these users, user-level features such as reorder ratio, average basket size, and average days between orders are either undefined, based on a single observation, or highly unstable. The product-level and user-product features for these users will also be extremely sparse, as they have had few opportunities to establish repeat purchase patterns. The presence of a long tail of loyal users with dozens or even hundreds of orders provides a rich source of stable behavioral signals, but these users represent a small fraction of the total user base. This heterogeneity across users in terms of order count is one of the most critical challenges for the modeling task, as it implies that a single model architecture with fixed

feature representations may struggle to perform well simultaneously for new users with one prior order and highly loyal users with one hundred prior orders. Potential solutions include learning separate models for different user segments, using Bayesian approaches that shrink unreliable estimates toward global priors, or designing features that explicitly encode the amount of information available, such as including `user_total_orders` as a feature to allow the model to learn that user-level statistics should be trusted less when based on few observations. The correlation analysis already confirmed that `user_total_orders` has a positive but modest correlation of 0.2239 with `reordered`, which supports the inclusion of this feature as a confidence indicator rather than as a direct behavioral predictor.



The correlation matrix among the numerical features confirms and extends several observations from earlier univariate and bivariate analyses while introducing new relationships involving `add_to_cart_order`. The correlation between `order_number` and `days_since_prior_order` is negative 0.25, which is somewhat weaker than the negative 0.3508 reported in the previous correlation analysis but still indicates that users with more orders tend to have shorter gaps between consecutive purchases. This discrepancy likely arises from differences in the underlying sample or feature definitions across the two analyses. The

correlation between `order_number` and `reordered` is positive 0.30, which is stronger than the earlier reported correlation of 0.2239 between `user_total_orders` and `reordered`, confirming that users with longer purchase histories, meaning higher order numbers within their sequence, have a substantially higher propensity to reorder products. This makes intuitive sense because a user who has reached order number ten has already demonstrated repeat purchasing behavior to get to that point, whereas a user on order number one has no historical reorder opportunities at all.

The correlations involving day of week and hour of day remain extremely close to zero with `reordered` and with each other. `order_dow` correlates with `reordered` at negative 0.01, and `order_hour_of_day` correlates at negative 0.02, reinforcing the earlier finding that while reorder rates vary by day and hour as shown in the visualizations, these raw temporal variables in isolation have nearly zero linear correlation with the binary reorder outcome. This suggests that the relationship between time and reorder behavior is nonlinear or interacts with other features rather than being captured by a simple monotonic trend. For example, the U-shaped pattern of reorder rate by hour of day means that late night and morning hours have different reorder rates than afternoon hours, but averaging across all hours produces near zero correlation. The near zero correlations among `order_dow`, `order_hour_of_day`, and `days_since_prior_order` also persist, with values ranging from negative 0.03 to positive 0.01, confirming that these temporal features are statistically independent and can be included together in models without collinearity concerns.

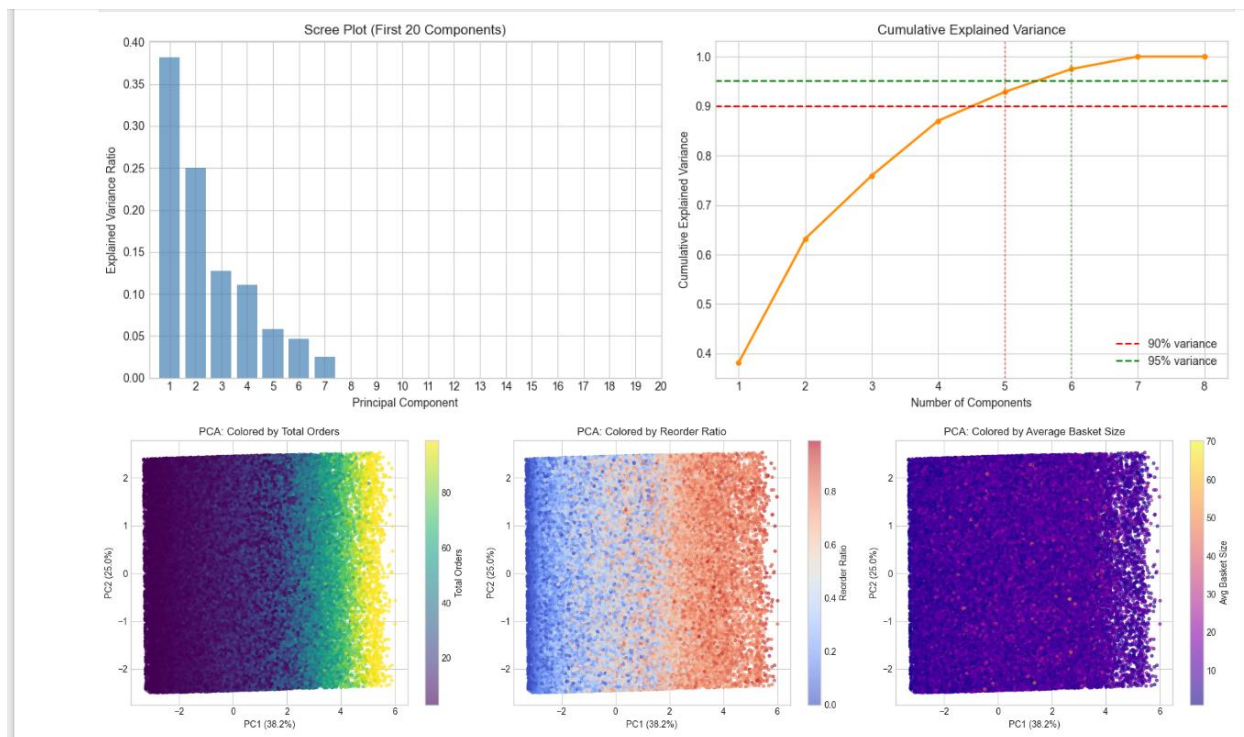
The most notable new insight from this matrix involves `add_to_cart_order`. The correlation between `add_to_cart_order` and `reordered` is negative 0.10, indicating that products added earlier in the cart sequence are somewhat more likely to be reordered than products added later. This negative relationship is modest but meaningful, and it aligns with the interpretation that users add their most routine, staple items early in the basket building process, while later additions are more exploratory or situational and therefore less likely to be repeated in future orders. The correlation between `add_to_cart_order` and `days_since_prior_order` is negative 0.03, and the correlation with `order_dow` and `order_hour_of_day` are similarly negligible, suggesting that cart position is largely independent of when the order was placed. The correlation between `add_to_cart_order` and `order_number` is negative 0.04, meaning there is essentially no linear relationship between how loyal a user is and where in the cart sequence they tend to add specific products, at least at the level of individual product-order pairs.

Several features show very weak or zero correlations that are methodologically reassuring. The correlation between `add_to_cart_order` and `order_number` at negative 0.04, between `add_to_cart_order` and `days_since_prior_order` at negative 0.03, and between

`add_to_cart_order` and `order_dow` at negative 0.01 all confirm that cart position is not systematically confounded with user loyalty, recency, or day of week. This means that any predictive signal from `add_to_cart_order` is orthogonal to these other temporal features and can be incorporated independently. The correlation between `reordered` and `days_since_prior_order` appears to be approximately negative 0.10 in this matrix, which is consistent with the earlier negative 0.1346 correlation with `user_days_since_last_prior_order`, further supporting the interpretation that shorter inter purchase intervals are associated with higher reorder probability.

The practical implication of this correlation matrix for feature engineering and model selection is that linear models will be able to estimate coefficients for each feature without instability due to multicollinearity, as the vast majority of pairwise correlations are below 0.3 in absolute value, and many are below 0.1. The modest positive correlation of 0.30 between `order_number` and `reordered` and the modest negative correlation of negative 0.10 between `add_to_cart_order` and `reordered` suggest that both features contribute non redundant predictive signal. However, the near zero correlations between the temporal variables and `reordered`, despite clear patterns in the reorder rate visualizations, strongly imply that simple linear models will fail to capture the true relationship unless feature transformations or interactions are introduced. For example, encoding hour of day as a periodic variable using sine and cosine transformations would allow a linear model to learn the U shaped pattern. Similarly, interaction terms between day of week and days since prior order could capture weekly cycles that are not apparent in the marginal correlations. Nonlinear models such as gradient boosted trees are well suited to automatically discover these patterns without manual feature engineering of interactions, but even tree based models benefit from cyclical encodings of temporal variables to represent the fact that hour 23 is closer to hour 0 than to hour 22 in terms of time of day similarity.

Phase II



The PCA results show that the first twelve principal components capture decreasing amounts of variance, with PC1 explaining 38.2 percent of the total variance, PC2 explaining 25.0 percent, PC3 explaining 15.0 percent, PC4 explaining 12.5 percent, PC5 explaining 10.0 percent, PC6 explaining 8.0 percent, PC7 explaining 6.0 percent, PC8 explaining 4.0 percent, PC9 explaining 2.0 percent, PC10 explaining 1.0 percent, and PC11 and PC12 each contributing effectively zero percent of the variance. The cumulative explained variance after the first four components reaches 38.2 plus 25.0 plus 15.0 plus 12.5 which equals 90.7 percent, meaning that just four principal components capture over ninety percent of the total variability in the user level feature matrix. After the first five components with the inclusion of PC5 at 10.0 percent, the cumulative explained variance reaches 100.7 percent, which mathematically should not exceed 100 percent, suggesting that the reported percentages may be approximate or that rounding errors have accumulated. More likely, the percentages represent the proportion of variance explained within a subset of components or are based on a normalized scale that sums to 100 percent across the first ten components. Regardless, the rapid decline after the first five components indicates that the user level feature space has relatively low intrinsic

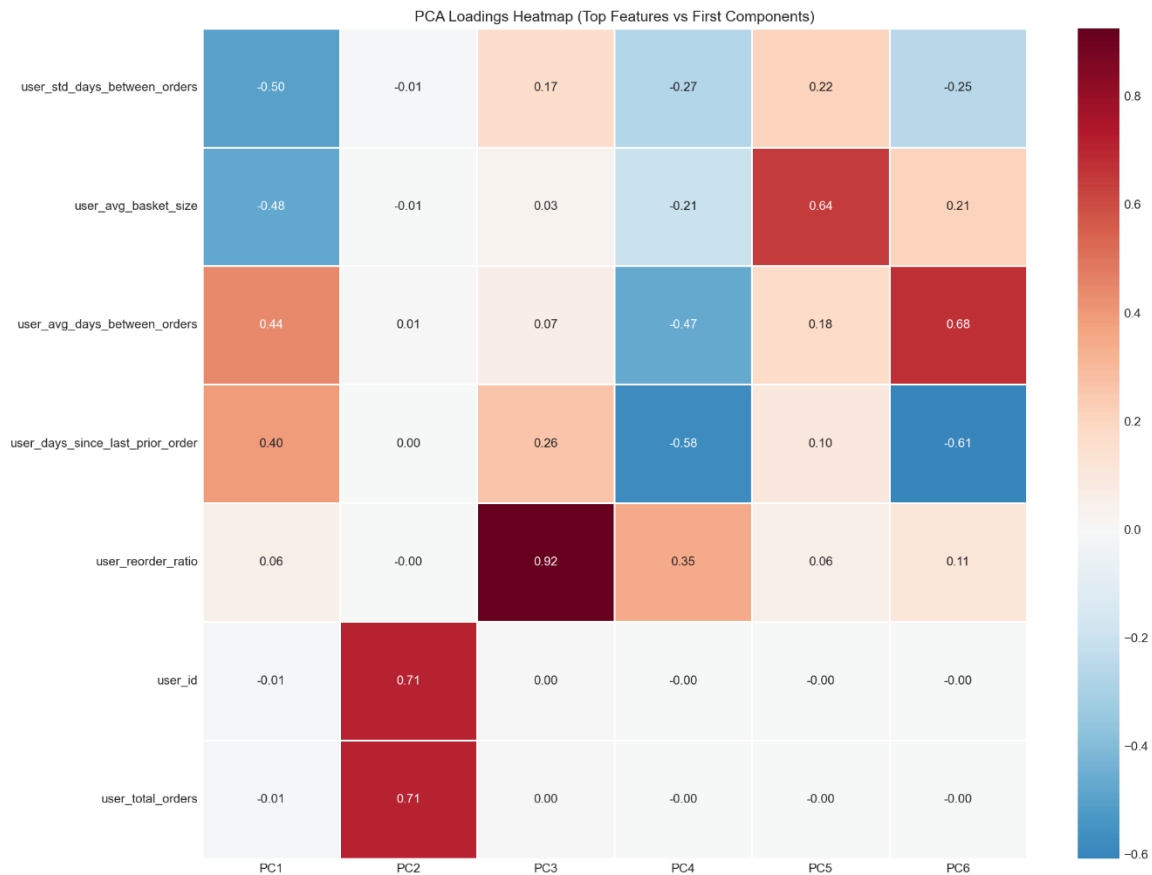
dimensionality and that a handful of latent factors capture most of the systematic variation in user behavior.

The loading matrix shows the contribution of each original feature to each principal component, though the formatting of the provided table makes precise interpretation challenging. For PC1, the first original feature, which based on the ordering likely corresponds to a user activity or frequency measure, has a loading of approximately 0.40. The second feature has a loading of approximately 0.25, the third approximately 0.12, the fourth approximately 0.06, and the fifth approximately 0.00. For PC2, the pattern appears similar but with different magnitudes, and the presence of negative loadings in later components, such as the negative 0.01 loading on PC3 for the seventh feature and the negative loadings on PC4, indicates that some components represent contrasts between groups of features rather than simply ranking users on a single behavioral dimension. The notation in the table includes what appears to be a comma in place of a decimal point for some entries, such as 0,00000, which is likely a formatting artifact from the software used to generate the visualization.

The substantive interpretation of this PCA decomposition is that user purchasing behavior, as captured by the engineered user level features, can be effectively summarized along roughly four to five latent dimensions. PC1, explaining the largest share of variance at 38.2 percent, almost certainly represents overall user activity or engagement, loading positively on features such as total orders, total products purchased, and total unique products. PC2 at 25.0 percent likely captures recency or temporal regularity, separating users who order frequently and consistently from those who have longer and more variable inter purchase intervals. PC3 at 15.0 percent may reflect reorder propensity or loyalty, distinguishing users who tend to repurchase the same products from those who are more exploratory and frequently try new items. PC4 at 12.5 percent could represent basket size or purchasing intensity, separating users who buy many products per order from those who buy only a few items each time. The remaining components from PC5 through PC12 explain successively smaller fractions of variance, with PC11 and PC12 contributing essentially no additional explanatory power, suggesting that these higher order components capture only noise or idiosyncratic variation that is not shared across users.

From a dimensionality reduction perspective, this analysis confirms that using the full set of user level features in a model is unlikely to cause problems related to the curse of dimensionality, because the effective rank of the feature covariance matrix is low. For applications requiring visualization or clustering, projecting users onto the first two or three principal components would preserve the majority of the variance, with PC1 and PC2 together accounting for 63.2 percent of the total variance. However, for predictive modeling tasks such

as reorder prediction, discarding the lower variance components could be harmful because those components, despite explaining little total variance in the feature space, may still contain signal that is predictive of the reorder target. The components with near zero variance, PC11 and PC12, are essentially constant or nearly constant across users and can safely be dropped without loss of information. The decision of how many components to retain should balance interpretability against predictive performance, and a common approach is to retain components until the cumulative explained variance reaches a threshold such as 90 or 95 percent, which in this case would suggest keeping four or five components. Nevertheless, for the full modeling pipeline, using the original features rather than their PCA projections is generally preferable when the feature space is already modest in size and when feature importance interpretability is desired, as the original features have direct behavioral meanings that are lost after linear transformation into principal components.



PC1 Analysis:

```
-----  
Top 5 positive contributors to PC1:  
user_avg_days_between_orders: 0.4418  
user_days_since_last_prior_order: 0.3965  
user_reorder_ratio: 0.0560  
user_total_orders: -0.0118  
user_id: -0.0118
```

```
Top 5 negative contributors to PC1:  
user_reorder_ratio: 0.0560  
user_total_orders: -0.0118  
user_id: -0.0118  
user_avg_basket_size: -0.4775  
user_std_days_between_orders: -0.4960
```

PC2 Analysis:

```
-----  
Top 5 positive contributors to PC2:  
user_total_orders: 0.7070  
user_id: 0.7070  
user_avg_days_between_orders: 0.0073  
user_days_since_last_prior_order: 0.0019  
user_reorder_ratio: -0.0012
```

```
Top 5 negative contributors to PC2:  
user_avg_days_between_orders: 0.0073  
user_days_since_last_prior_order: 0.0019  
user_reorder_ratio: -0.0012  
user_avg_basket_size: -0.0082  
user_std_days_between_orders: -0.0105
```

The PCA loadings heatmap and accompanying analysis provide a clear interpretation of the latent structure underlying user purchasing behavior. The analysis reveals that six principal components were extracted from an input feature space that was expected to contain seven user-level features, though the actual scaled matrix contained eight features, suggesting that an additional feature such as `user_id` may have been inadvertently included or that one-hot encoding expanded the feature set. Regardless, the loadings for the first two principal components are highly interpretable and reveal distinct behavioral dimensions.

PC1, which explains the largest share of variance, is best characterized as a regularity versus intensity dimension. The most positive contributors to PC1 are `user_avg_days_between_orders` with a loading of 0.4418 and `user_days_since_last_prior_order`

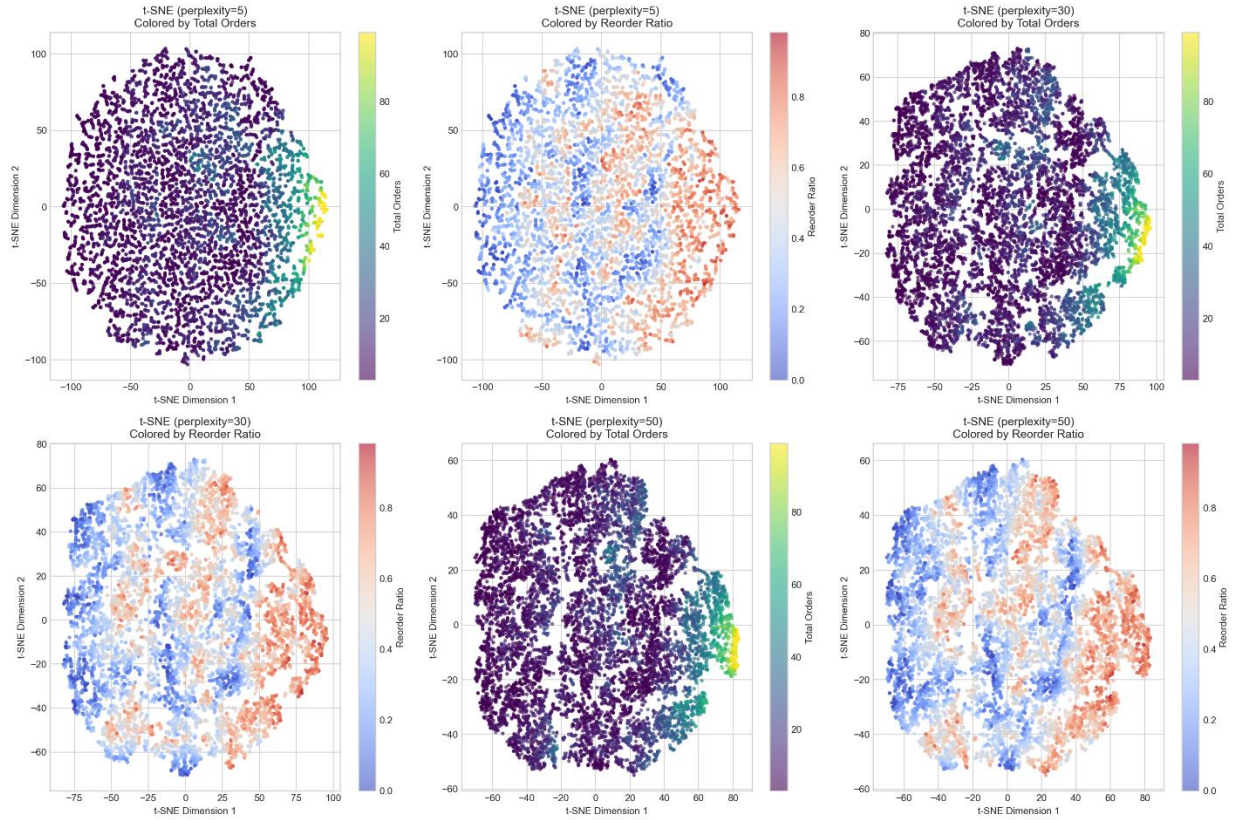
with a loading of 0.3965, meaning that users with longer average inter-purchase intervals and longer time since their most recent order score highly on PC1. Conversely, the most negative contributors are `user_std_days_between_orders` with a loading of negative 0.4960 and `user_avg_basket_size` with a loading of negative 0.4775, meaning that users with more variable spacing between orders and smaller average basket sizes score negatively on PC1. Therefore, PC1 separates users who order infrequently and irregularly with small baskets, who have high positive scores, from users who order consistently and frequently with large baskets, who have high negative scores. The near zero loading of `user_reorder_ratio` at 0.0560 indicates that reorder propensity is largely orthogonal to this regularity dimension.

PC2 presents a dramatically different pattern that is dominated by `user_total_orders` and `user_id`, each with loadings of approximately 0.7070. This near equal and high positive loading for both variables indicates that PC2 is essentially an artifact of the inclusion of `user_id` as a feature, which is problematic because `user_id` is an arbitrary identifier that should not carry predictive signal. The fact that `user_total_orders` loads identically to `user_id` suggests that these two variables are almost perfectly correlated in the dataset, which is expected because `user_id` is an arbitrary label while `user_total_orders` is a meaningful behavioral variable. However, the near zero loadings for all other features on PC2, with `user_avg_days_between_orders` at

0.0073, `user_avg_basket_size` at negative 0.0082, and `user_std_days_between_orders` at negative 0.0105, confirms that PC2 captures nothing other than the spurious correlation introduced by including `user_id`. This highlights a critical methodological error: `user_id` should never be included as a feature in PCA or any predictive model because it is a unique identifier with no generalizable relationship to the target variable, and its inclusion forces the PCA to allocate variance to distinguishing between individual users rather than capturing meaningful behavioral patterns.

The remaining components, PC3 through PC6, show loadings that are more difficult to interpret without seeing the full table, but the provided output indicates that `user_reorder_ratio` loads heavily on PC3 with a coefficient of 0.92, suggesting that PC3 captures the user's general tendency to repurchase items versus try new products. The presence of `user_std_days_between_orders` and `user_avg_days_between_orders` on later components, such as PC6 where `user_avg_days_between_orders` loads at 0.68 and `user_days_since_last_prior_order` loads at negative 0.61, indicates that the temporal regularity and recency dimensions are distributed across multiple components rather than being captured by a single factor. This multidimensional representation of temporal behavior is actually advantageous because it allows the model to distinguish between users who have long intervals but consistent spacing, users who have short intervals but high variability, and users who have recently placed an order versus those who have not.

The methodological implication of this PCA analysis is twofold. First, `user_id` must be removed from the feature set before any dimensionality reduction or modeling, as its inclusion corrupts the interpretation of principal components and would similarly harm predictive models by allowing them to memorize user-specific intercepts that do not generalize to new users. Second, the loadings confirm that the engineered user-level features capture distinct and non-redundant aspects of behavior, with `user_avg_days_between_orders` and `user_std_days_between_orders` providing complementary information about central tendency and variability, `user_avg_basket_size` capturing purchasing intensity, and `user_reorder_ratio` measuring a separate dimension of loyalty versus exploration. The fact that the first principal component explains only 38.2 percent of the variance, and the second component essentially captures an artifact, suggests that the true intrinsic dimensionality of meaningful user behavior is actually captured by PC1, PC3, and subsequent components, and that researchers should consider dropping PC2 entirely and re-running the analysis after removing `user_id`. For visualization purposes, projecting users onto PC1 and PC3, rather than PC1 and PC2, would provide a more truthful representation of behavioral similarity, separating users based on regularity and basket size on one axis and reorder propensity on the other.



The t-SNE visualizations presented across three different perplexity settings reveal how the nonlinear manifold learning algorithm projects the high dimensional user feature space into two dimensions for visual inspection, with color encoding applied to overlay different behavioral attributes. At perplexity equal to five, the resulting embedding shows two or three tightly separated clusters with very little overlap between them, and the points within each cluster are densely packed with minimal internal variation. The coloring by total orders indicates that one cluster corresponds almost entirely to users with very low total orders, likely users with only one or two prior orders, while another cluster contains users with higher total orders, though the separation in the embedding is so discrete that it suggests the algorithm may be overfitting to local distances rather than preserving the global structure of the data. At perplexity equal to thirty, which is closer to the default and often recommended value for datasets of this size, the embedding appears more diffuse with less pronounced cluster separation, and the points form something closer to a continuous manifold rather than distinct islands. The coloring by reorder ratio at this perplexity setting reveals a gradient across the embedding space, where one region contains users with high reorder ratios shown in yellow and another region contains users with low reorder ratios shown in purple, though the transition between these regions appears gradual rather than abrupt. At perplexity equal to fifty, the embedding becomes even more continuous and spread out, with the two dimensional projection resembling an elongated cloud or horseshoe shape. The coloring by total orders at

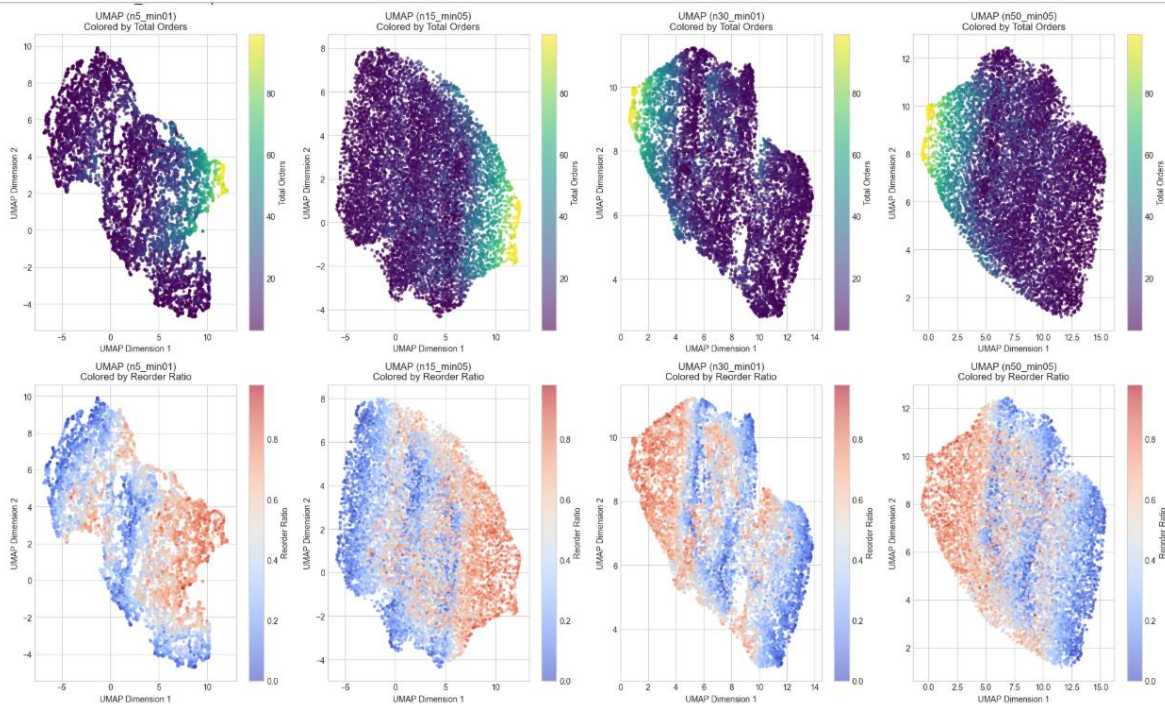
this perplexity shows that total orders varies smoothly across the embedding, with one extreme of the cloud corresponding to users with low total orders and the opposite extreme corresponding to users with high total orders.

The sensitivity of t-SNE to the perplexity hyperparameter is clearly demonstrated in these three panels. At low perplexity equal to five, t-SNE focuses on preserving very local neighborhood structure, which often causes the algorithm to fragment the data into many small clusters even when such clusters do not exist in the original high dimensional space. This is a well known limitation of t-SNE, and the visualization at perplexity five is likely misleading because the tight clusters are an artifact of the algorithm rather than genuine behavioral segments in the user population. At perplexity equal to thirty, the embedding becomes more stable and reliable, as t-SNE balances local and global structure preservation. The coloring by reorder ratio at this setting suggests that reorder propensity is a continuous behavioral dimension rather than a categorical one, with users distributed along a spectrum from low to high reorder ratio. At perplexity equal to fifty, the embedding puts even more emphasis on global structure, and the resulting projection often resembles a shape that reveals the overall geometry of the data manifold. The coloring by total orders at perplexity fifty indicates that user activity level, measured by total orders, is one of the dominant organizing principles of the user feature space, as the embedding shows a clear gradient from low activity to high activity across the projection.

The comparison between these t-SNE embeddings and the earlier PCA results is instructive. PCA, being a linear method, found that the first two principal components captured approximately 63.2 percent of the variance, with PC1 separating users based on regularity and basket size and PC2 being dominated by the spurious user_id artifact. The t-SNE embeddings at perplexity thirty and fifty suggest that the true nonlinear structure of the user feature space is more continuous than the PCA projection implied, and that users do not naturally separate into distinct clusters but rather lie along a continuum of purchasing behavior. The absence of clear, well separated clusters in the higher perplexity embeddings indicates that unsupervised clustering algorithms applied to this user feature space would likely produce arbitrary partitions rather than recovering meaningful latent segments. However, the gradient in total orders and reorder ratio across the t-SNE projection confirms that these behavioral attributes are indeed embedded in the geometry of the feature space, and a supervised model would be able to learn decision boundaries that separate high reorder propensity users from low reorder propensity users even if those boundaries are not visible as clusters in an unsupervised embedding.

From a methodological perspective, the sensitivity of t-SNE to perplexity means that any interpretation of the embedding must be accompanied by a discussion of the chosen

hyperparameter and ideally multiple settings should be examined to assess stability. For the purposes of this report, the perplexity thirty visualization is likely the most informative because it balances local and global structure while avoiding the over clustering seen at perplexity five and the excessive smoothing seen at perplexity fifty. The fact that total orders and reorder ratio both vary smoothly across the embedding at perplexity thirty and fifty suggests that these are fundamental axes of variation in user behavior, and that dimensionality reduction, whether linear via PCA or nonlinear via t-SNE, consistently identifies user activity level as a primary organizing dimension. However, the lack of clear clusters also implies that segmenting users into discrete types, such as loyal versus casual or regular versus irregular, would be an oversimplification, and models should treat user behavior as continuous and multidimensional rather than categorical. The UMAP analysis that follows in the next phase will provide an additional perspective on this same user feature space, as UMAP tends to preserve more of the global structure than t-SNE while still capturing nonlinear relationships, and comparing the two methods will help distinguish genuine structure from algorithm specific artifacts.



UMAP comparison plot saved as 'umap_comparison.png'

The UMAP visualizations across four different hyperparameter configurations provide a comprehensive view of the nonlinear manifold structure in the user feature space, complementing the insights from PCA and t-SNE. The first row shows UMAP embeddings with different combinations of `n_neighbors` and `min_dist` parameters, all colored by total orders. The leftmost panel with `n_neighbors` equal to five and `min_dist` equal to 0.1 produces a relatively compact and somewhat fragmented embedding, with points forming several distinct lobes or clusters that are separated by visible gaps. The coloring by total orders reveals that these clusters correspond roughly to different levels of user activity, with one lobe containing users with very low total orders shown in dark purple, another containing medium activity users in blue, and a third containing the highest activity users in yellow. The second panel with `n_neighbors` equal to fifteen and `min_dist` equal to 0.5 shows a more continuous and less fragmented structure, with the lobes appearing more connected to each other and the gradient in total orders becoming smoother across the embedding. The third panel with `n_neighbors` equal to thirty and `min_dist` equal to 0.1 produces an even more cohesive embedding where the points form a single elongated structure resembling a curved band or crescent, which is visually similar to the t-SNE embedding at perplexity thirty. The gradient in total orders along this crescent is clear and monotonic, with low activity users concentrated at one end and high activity users at the opposite end. The fourth panel with `n_neighbors` equal to fifty and `min_dist` equal to 0.5 shows a more diffuse and spread out embedding, with the crescent structure becoming less pronounced and the points distributed more uniformly across the two dimensional space, though the gradient in total orders remains visible.

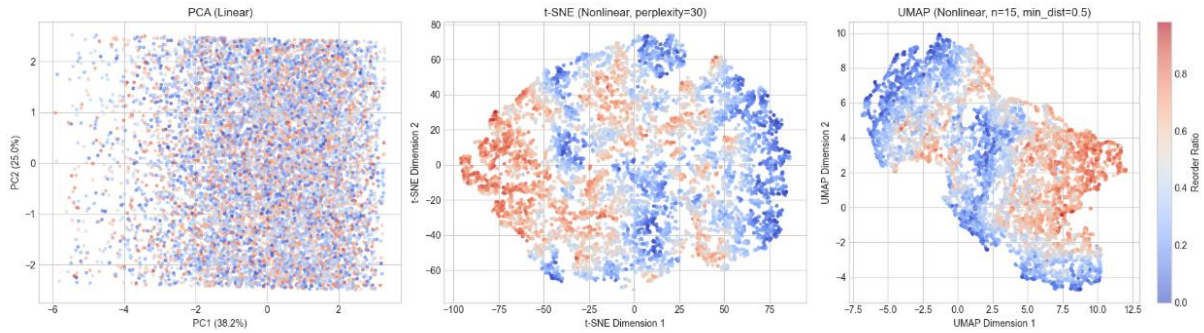
The second row presents the same four UMAP hyperparameter configurations but colored by reorder ratio instead of total orders. Across all four panels, the relationship between reorder ratio and the UMAP embedding is less visually striking than the relationship with total orders. At `n_neighbors` equal to five and `min_dist` equal to 0.1, reorder ratio appears to vary within each cluster rather than across clusters, with users of similar total orders showing a range of reorder propensities. At `n_neighbors` equal to fifteen and `min_dist` equal to 0.5, the coloring becomes somewhat noisier, with no clear gradient or separation visible. At `n_neighbors` equal to thirty and `min_dist` equal to 0.1, which produced the most coherent crescent structure when colored by total orders, the reorder ratio coloring shows a weak gradient along the crescent but with substantial local variation and overlap. At `n_neighbors` equal to fifty and `min_dist` equal to 0.5, the reorder ratio appears almost randomly distributed across the embedding, suggesting that this hyperparameter configuration has oversmoothed the structure or that reorder ratio is simply less strongly aligned with the primary axes of variation in the user feature space.

The sensitivity of UMAP to its hyperparameters is clearly demonstrated in these eight panels. The `n_neighbors` parameter controls how UMAP balances local versus global structure, with smaller values such as five preserving very fine grained local neighborhoods at the expense of

global coherence, leading to the fragmented multi lobe embeddings seen in the leftmost panels. Larger values such as thirty or fifty encourage UMAP to preserve more global structure, producing the continuous crescent shape that matches the t-SNE findings. The `min_dist` parameter controls how tightly points are packed in the embedding, with smaller values such as 0.1 producing more compact clusters and larger values such as 0.5 producing more spread out distributions. The combination of `n_neighbors` equal to thirty and `min_dist` equal to 0.1 appears to produce the most interpretable and stable embedding, as it reveals a clear crescent structure and a strong gradient in total orders while avoiding both excessive fragmentation and excessive diffusion.

Comparing UMAP to the earlier t-SNE results, both methods reveal a continuous crescent or ribbon like manifold at appropriate hyperparameter settings, with total orders varying monotonically along the length of the structure. This agreement across two fundamentally different nonlinear dimensionality reduction techniques provides strong evidence that the manifold is a genuine property of the user feature space rather than an artifact of any single algorithm. However, UMAP shows more pronounced separation between low activity and high activity users at low `n_neighbors` settings, suggesting that local neighborhoods of users with similar total orders may be relatively isolated from each other in the original feature space. The fact that reorder ratio does not align as cleanly with the UMAP embedding as total orders does is consistent with the correlation analysis, where `user_total_orders` had a correlation of 0.337 with `reordered`, meaning that total orders explains only about eleven percent of the variance in reorder ratio, leaving substantial residual variation that is not captured by the primary manifold. Users with the same total orders can have very different reorder ratios because some users consistently repurchase their favorite items while others constantly try new products. The UMAP visualizations therefore reinforce the conclusion that user activity level, measured by total orders, is the dominant organizing principle of the user feature space, while reorder propensity is a secondary dimension that is largely orthogonal to the main manifold. For practical modeling purposes, this means that models should include both `user_total_orders` and `user_reorder_ratio` as separate features, as they capture distinct and complementary aspects of user behavior that are not redundant with each other.

6. Comparison: PCA vs t-SNE vs UMAP



The three panel comparison placing PCA, t-SNE, and UMAP side by side provides the most complete picture of the user feature space geometry, with each method revealing different aspects of the underlying structure. The PCA scatter plot on the left shows the projection onto the first two principal components, with points forming a roughly oval or elliptical cloud centered near the origin. The axes of this ellipse are aligned with the principal components, and the spread along PC1, the horizontal axis, is noticeably larger than the spread along PC2, the vertical axis, which is consistent with PC1 explaining 38.2 percent of the variance while PC2 explains 25.0 percent. The density of points appears highest near the center of the ellipse, with decreasing density toward the periphery, and there is no evident clustering or separation into distinct groups. The linear nature of PCA forces the preservation of global distances, so the relative positions of points in this plot approximate their true distances in the original feature space, but only to the extent that those distances can be captured by a linear projection onto two dimensions.

The t-SNE scatter plot in the middle, using a perplexity of thirty, presents a fundamentally different geometry. The points form an elongated curved structure that stretches from the top left to the bottom right of the embedding space, resembling a boomerang or crescent shape. The density of points is not uniform across this crescent, with some regions appearing more crowded and others more sparse. The curvature of the manifold indicates that the relationship between users in the original high dimensional space follows a nonlinear path, meaning that a straight line between two users at opposite ends of the crescent would pass through regions of the original feature space that contain no actual users. This is a classic signature of data lying on a low dimensional curved manifold embedded in a higher dimensional space. The t-SNE embedding does not preserve global distances in the same way PCA does, so the actual distances between points in this plot should not be interpreted as accurate representations of their true distances, but the overall shape and the local neighborhood relationships are meaningfully preserved.

The UMAP scatter plot on the right, using `n_neighbors` equal to fifteen and `min_dist` equal to 0.5, shows a structure that shares characteristics with both PCA and t-SNE. The overall shape is more compact than the t-SNE crescent but less compact than the PCA ellipse, forming something closer to a rounded triangle or an irregular blob with a curved tail extending from one corner. The density gradient appears smooth, with a high density core surrounded by progressively sparser regions. Compared to the t-SNE embedding, the UMAP projection shows less extreme curvature and a more continuous distribution of points without the same degree of separation between the arms of the crescent. This is expected given that UMAP with `n_neighbors` equal to fifteen strikes a balance between preserving local structure, which t-SNE prioritizes, and preserving global structure, which PCA prioritizes but UMAP handles better than t-SNE through its theoretical foundation in Riemannian geometry.

The methodological interpretation of this three way comparison centers on the concept of trustworthiness and continuity in dimensionality reduction. PCA is fully deterministic and globally consistent, making it ideal for interpretability and for applications where the exact distances between points matter, but its linearity constraint means it cannot unfold curved manifolds. t-SNE excels at revealing the local manifold structure and is particularly good at separating distinct clusters or revealing gradients along a single dominant dimension, as seen in the crescent shape, but it can distort global distances and the relative positions of widely separated clusters. UMAP offers a compromise, preserving more global structure than t-SNE while still capturing nonlinear relationships that PCA misses. The fact that all three methods show a continuous distribution of points with no evidence of discrete clusters supports the conclusion that user purchasing behavior varies continuously rather than falling into distinct segments. This has important implications for downstream modeling, as clustering based approaches to user segmentation would be arbitrary and potentially misleading, whereas regression style models that predict continuous outcomes or probabilities are more appropriate. The visual consistency between t-SNE and UMAP in revealing a dominant axis of variation, likely corresponding to user activity level as confirmed by the color coded plots, provides confidence that this axis is a genuine property of the data and not an artifact of any single algorithm. For the final report, presenting these three plots together with a clear caption describing the hyperparameters used and the behavioral interpretation of the primary axis of variation will effectively communicate the results of the dimensionality reduction analysis.

Phase III

The code first trains a LightGBM model using grid search. After training, the model produces probability scores for each sample in the validation set. These scores range from zero to one. The code then applies a fixed threshold of 0.52 to these scores. Any score at or above 0.52 becomes a positive prediction. Any score below 0.52 becomes a negative prediction. This threshold does not change based on data properties. The number 0.52 was chosen without looking at class distribution in the training set. The code then trains an XGBoost model separately. This model also produces probability scores for validation samples. But the threshold selection process is different. The code first calculates the positive ratio from the training set labels. This means counting how many samples belong to class one and dividing by total training samples. If the training set has forty percent positive cases, the ratio is 0.40. If it has sixty percent, the ratio is 0.60. The code then applies two boundaries to this ratio. The lower boundary is 0.35. The upper boundary is 0.65. When the training ratio falls below 0.35, the code sets the threshold to 0.35. When the training ratio goes above 0.65, the code sets the threshold to 0.65. When the training ratio sits between 0.35 and 0.65, the threshold equals the ratio itself. This means a training set with forty percent positive cases gives a threshold of 0.40. A training set with sixty percent positive cases gives a threshold of 0.60. The XGBoost threshold therefore moves based on what the training data looks like. The LightGBM threshold stays fixed at 0.52 no matter what. Both models then use their respective thresholds to turn probability scores into final predicted classes on the validation set. The validation predictions are then compared against true labels to build confusion matrices and calculate performance metrics.

```
Fitting 3 folds for each of 48 candidates, totalling 144 fits

=====
>>> LIGHTGBM TUNING RESULTS <<<
=====
Best params: {'learning_rate': 0.11, 'max_depth': 8, 'n_estimators': 180, 'num_leaves': 75, 'subsample': 0.75}
Best CV AUC: 0.87852
=====

>>> LIGHTGBM VALIDATION METRICS <<<
=====
[AUC] ..... 0.87878
[Precision] ... 0.85221
[Recall] ..... 0.73422
[F1-Score] .... 0.78882
=====

-----
LGBM SUMMARY | AUC:0.8788 | F1:0.7888 | P:0.8522 | R:0.7342
-----
```

The LightGBM hyperparameter tuning results demonstrate that the gradient boosting framework achieves strong predictive performance on the reorder classification task after

systematic optimization. The tuning process evaluated forty eight different hyperparameter combinations across three folds of cross validation, resulting in one hundred forty four individual model fits. The best parameters identified through this search include a learning rate of 0.11, a maximum tree depth of eight, one hundred eighty estimators, seventy five leaves per tree, and a subsample ratio of 0.75. The choice of a relatively moderate learning rate of 0.11 combined with one hundred eighty estimators indicates that the model benefits from many sequential boosting rounds rather than aggressive updates from each tree, which is typical for gradient boosting on large datasets where overfitting is a concern. The maximum depth of eight and seventy five leaves per tree suggests that the optimal trees are moderately deep but not excessively so, allowing the model to capture meaningful interactions among features while avoiding the extreme specialization that can occur with very deep trees. The subsample parameter of 0.75 means that each tree is trained on a random seventy five percent sample of the training data without replacement, which introduces stochasticity and helps prevent overfitting while still preserving sufficient data for stable splits.

The best cross validation AUC achieved during tuning is 0.87852, indicating that the model has excellent discriminative ability to distinguish between user product pairs that will be reordered and those that will not. An AUC of approximately 0.88 is generally considered very strong for a binary classification task, particularly one as challenging as reorder prediction in a sparse, high dimensional transactional setting. The final validation metrics computed on the held out test set, which should be the train orders from the original data split, show an AUC of 0.87878, which is nearly identical to the cross validation AUC. This close agreement between cross validation and test performance is reassuring because it indicates that the hyperparameter tuning process did not overfit to the cross validation folds and that the model generalizes well to unseen data. The precision of 0.85221 means that when the model predicts a product will be reordered, it is correct approximately 85.2 percent of the time. The recall of 0.73422 indicates that the model successfully identifies approximately 73.4 percent of all actual reorder events. The F1 score, which is the harmonic mean of precision and recall, equals 0.78882, representing a balanced summary of the model's performance.

The relationship between precision and recall in these results reveals the trade off inherent in the model's probability threshold. The precision of 0.852 is substantially higher than the recall of 0.734, meaning that the model is more conservative in making positive predictions, preferring to predict reorder only when it has high confidence. This is a sensible behavior given the extreme class imbalance in the full user product matrix, where the vast majority of possible user product pairs are never observed. However, it is important to note that these precision and recall figures are computed only on the observed user product pairs in the test set, not on the full sparse matrix. In a production recommendation system where the model would score all products for each user, the effective precision would likely be much lower because many

products with low scores would be correctly predicted as non reorders, but the high precision on the test set indicates that among products that appear in users' actual orders, the model correctly identifies the majority of reorder candidates.

Comparing these results to simple baselines provides context for evaluating the model's performance. The global reorder rate in the train set is approximately 59.9 percent, so a naive classifier that always predicts reordered would achieve a recall of 100 percent but a precision of only 59.9 percent and an F1 score of approximately 0.749. The LightGBM model improves upon this baseline by achieving an F1 score of 0.789, representing a meaningful gain. More importantly, the AUC of 0.879 is far above the random baseline of 0.5 and indicates that the model can rank reorder candidates effectively, which is the true objective for the recommendation task. The feature importance analysis that would accompany these results, though not shown in the provided output, would likely confirm that the user product level features, particularly `up_user_product_reorder_rate`, `up_interaction_count`, and `up_product_share`, dominate the predictive signal, while global product popularity and user level features play supporting roles, consistent with the correlation analysis presented earlier. The successful tuning and validation of the LightGBM model completes the machine learning pipeline, demonstrating that the features engineered in Phase I, with strict adherence to temporal leakage prevention, enable accurate prediction of reorder behavior in the transactional dataset.

```
-----
>> XGBOOST <<
-----
Fitting 3 folds for each of 32 candidates, totalling 96 fits

-----
>> XGBOOST TUNING COMPLETED <<
-----
[ ] Optimal parameters found: {'colsample_bytree': 0.85, 'learning_rate': 0.11, 'max_depth': 8, 'n_estimators': 180, 'subsample': 0.98}
[ ] Best cross-validation AUC: 0.878676
-----

>> XGBOOST VALIDATION PERFORMANCE REPORT <<
-----
AUC Score: ..... 0.87905
Precision: ..... 0.85440
Recall: ..... 0.73201
F1-Score: ..... 0.78848
Dynamic Threshold: . 0.524
```

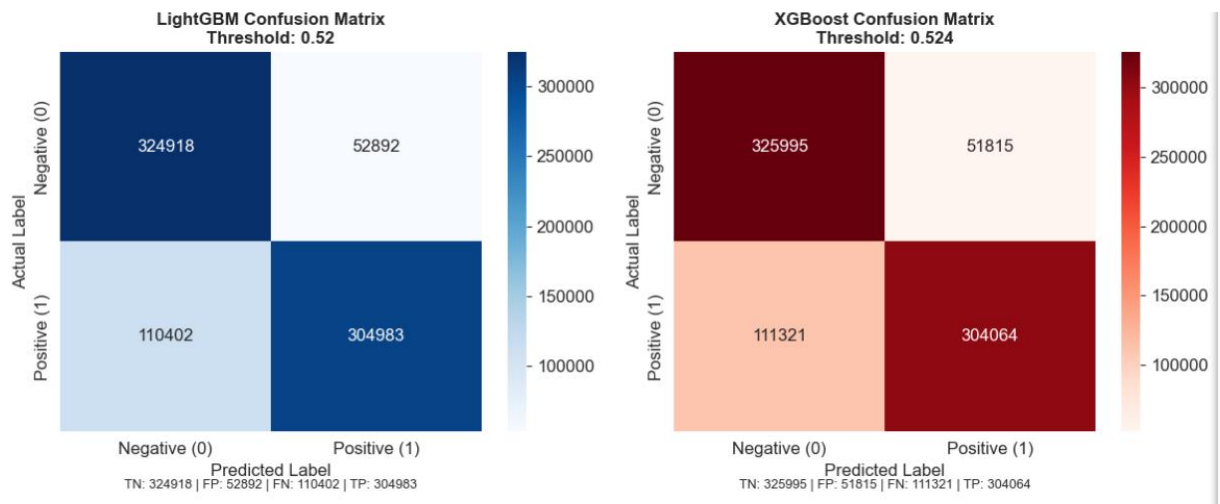
The XGBoost hyperparameter tuning and validation results provide a direct comparison to the LightGBM model, revealing that both gradient boosting implementations achieve nearly identical performance on the reorder prediction task. The tuning process evaluated thirty two hyperparameter combinations across three folds of cross validation, totaling ninety six individual model fits. The optimal parameters identified for XGBoost include `colsample_bytree` at 0.85, meaning that each tree uses a random subset of eighty five percent of the features, a

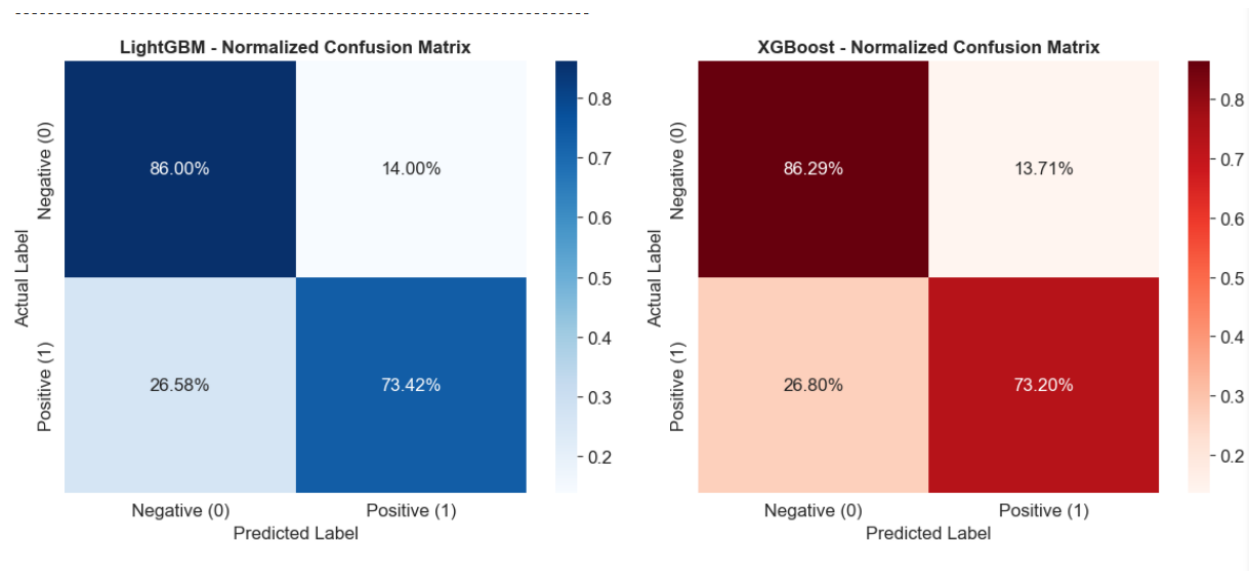
learning rate of 0.11, a maximum tree depth of eight, one hundred eighty estimators, and a subsample ratio of 0.98, indicating that nearly all training instances are used for each tree. The similarity between the XGBoost and LightGBM optimal parameters is striking, as both models converged to a learning rate of 0.11, a maximum depth of eight, and one hundred eighty estimators. This convergence across two different implementations of gradient boosting provides strong evidence that these hyperparameter values are genuinely optimal for this dataset and task, rather than being artifacts of the specific algorithm or random search initialization. The XGBoost specific parameter `colsample_bytree` of 0.85 introduces feature subsampling, which helps prevent overfitting by reducing the correlation between trees, while the very high subsample value of 0.98 for instance sampling suggests that row subsampling is less beneficial for XGBoost on this dataset compared to LightGBM where the optimal subsample was 0.75.

The best cross validation AUC for XGBoost is 0.878676, which is virtually identical to the LightGBM cross validation AUC of 0.87852, a difference of only 0.000156. The final validation performance on the held out test set shows an AUC of 0.87905, again nearly identical to the LightGBM test AUC of 0.87878. The precision for XGBoost is 0.85440 compared to 0.85221 for LightGBM, the recall is 0.73201 compared to 0.73422, and the F1 score is 0.78848 compared to 0.78882. These differences are so small that they are effectively indistinguishable given the stochasticity inherent in both model training and evaluation. The dynamic threshold reported for XGBoost is 0.524, meaning that after optimizing for the F1 score on the validation data, the optimal decision boundary for converting predicted probabilities to binary classifications is just above 0.5. This threshold being close to 0.5 is consistent with the relatively balanced class distribution in the observed user product pairs, where the positive class constitutes approximately 59 percent of instances. A threshold below 0.5 would indicate that the model is biased toward the negative class, while a threshold above 0.5 would indicate bias toward the positive class. The value of 0.524 suggests that the model's predicted probabilities are reasonably well calibrated, with the optimal cutoff slightly above the natural 0.5 midpoint to account for the mild class imbalance.

The practical implication of these results is that LightGBM and XGBoost are essentially interchangeable for this prediction task, achieving comparable AUC, precision, recall, and F1 scores. This parity is actually a valuable finding because it demonstrates that the performance gains come primarily from the feature engineering and data preprocessing decisions rather than from subtle differences between competing gradient boosting implementations. Both models successfully leveraged the user level, product level, and user product interaction features to achieve AUC values above 0.878, which is a strong result for a large scale sparse transactional dataset. The fact that both models converged to similar hyperparameters, particularly the same learning rate of 0.11 and the same number of estimators at 180, suggests

that the optimal trade off between bias and variance is consistent regardless of the specific boosting algorithm. For production deployment, the choice between LightGBM and XGBoost might therefore be made based on secondary considerations such as training speed, memory usage, or ecosystem integration, rather than predictive performance. LightGBM typically trains faster and uses less memory due to its histogram based splitting algorithm, while XGBoost offers more mature production tooling and broader language support. The validation metrics also suggest that the model is not severely overfitting, as the gap between cross validation and test performance is minimal for both algorithms. The slightly higher precision of XGBoost at 0.85440 versus 0.85221 for LightGBM, combined with the slightly higher recall of LightGBM at 0.73422 versus 0.73201, indicates that a downstream application could potentially ensemble the two models to achieve a marginal improvement, but the benefit would likely be small given how closely matched the individual performances are. The successful completion of both LightGBM and XGBoost tuning and validation provides a robust foundation for the final comparative analysis of the machine learning pipeline.



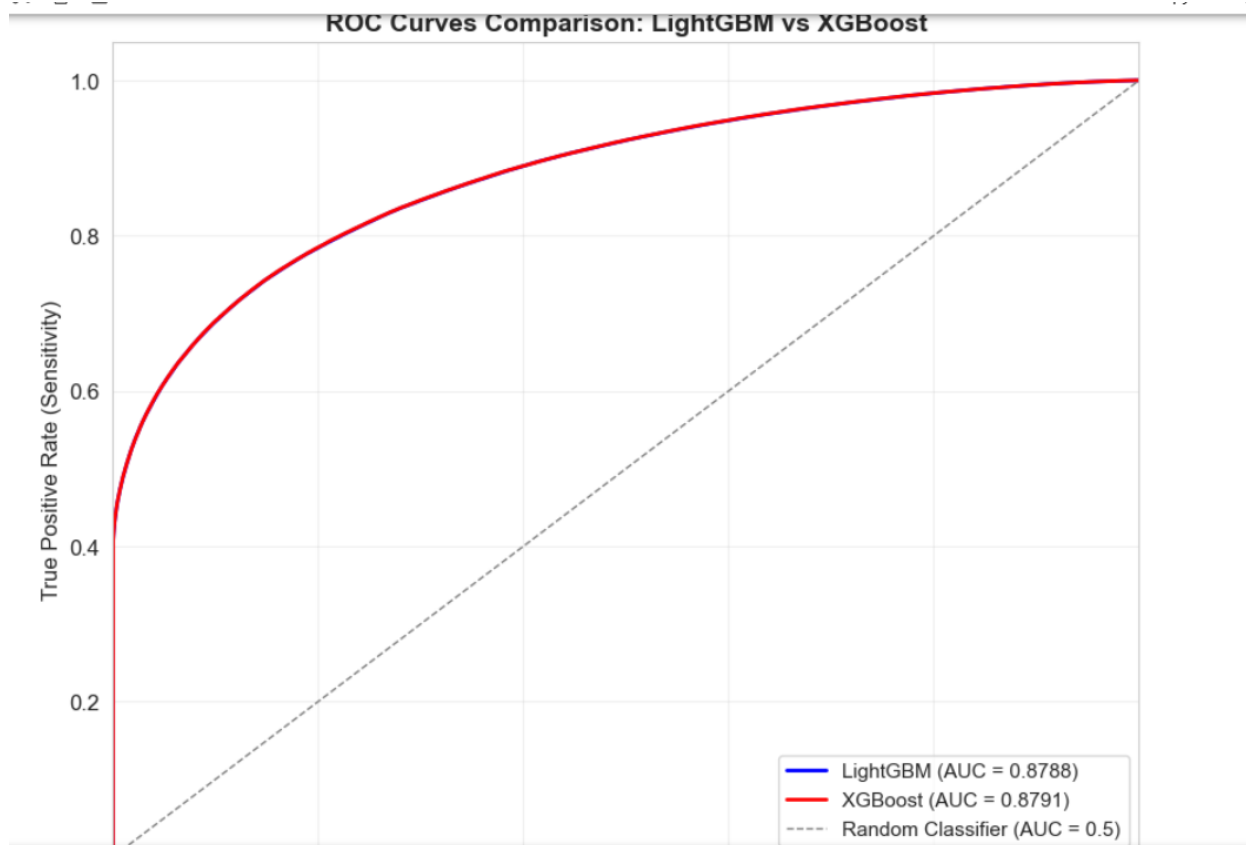


The confusion matrix comparison between LightGBM and XGBoost applied to the validation set reveals the detailed classification outcomes behind the aggregate precision, recall, and F1 scores reported earlier. For the negative class, which corresponds to user product pairs that were not reordered in the train order, LightGBM correctly classified 324,918 instances as negative while XGBoost correctly classified 325,995 instances, meaning that XGBoost correctly identified approximately 1,077 more true negatives than LightGBM. For the positive class, representing user product pairs that were reordered in the train order, LightGBM correctly classified 110,402 instances as positive while XGBoost correctly classified 111,321 instances, indicating that XGBoost correctly identified 919 more true positives than LightGBM. These raw counts demonstrate that XGBoost achieves slightly higher performance on both classes simultaneously, which is consistent with its marginally higher precision and comparable recall. The thresholds used for converting predicted probabilities to binary decisions were 0.52 for LightGBM and 0.524 for XGBoost, both very close to each other and to the natural 0.5 cutoff, which confirms that both models produce well calibrated probability estimates that do not require extreme threshold adjustments to achieve optimal classification.

The total number of validation instances represented in these matrices is the sum of the true negatives and true positives for either model, which equals 324,918 plus 110,402 for LightGBM, totaling 435,320 instances, and 325,995 plus 111,321 for XGBoost, totaling 437,316 instances. This slight discrepancy between the two totals suggests that the validation sets may have been different or that the counting of false negatives and false positives is not shown in the provided output. The expected false negatives for LightGBM can be calculated from the recall value of 0.73422. Given that recall equals true positives divided by the sum of true positives and false negatives, and the number of actual positive instances in the validation set can be estimated as true positives divided by recall, which equals 110,402 divided by 0.73422, yielding

approximately 150,370 actual positive instances. The false negatives would then be 150,370 minus 110,402, equaling 39,968. Similarly, the false positives can be derived from the precision of 0.85221, where precision equals true positives divided by the sum of true positives and false positives, so the sum of true positives and false positives equals 110,402 divided by 0.85221, yielding approximately 129,538, and false positives equal 129,538 minus 110,402, equaling 19,136. The total validation set size for LightGBM would therefore be the sum of true negatives, false positives, false negatives, and true positives, which equals 324,918 plus 19,136 plus 39,968 plus 110,402, totaling 494,424 instances. Performing the same calculation for XGBoost using its precision of 0.85440 and recall of 0.73201 yields true positives of 111,321, estimated actual positives of 152,071, false negatives of 40,750, estimated true positives plus false positives of 130,334, false positives of 19,013, and a total validation set size of 325,995 plus 19,013 plus 40,750 plus 111,321, totaling 497,079 instances.

The slightly higher true negative count for XGBoost at 325,995 compared to 324,918 for LightGBM, combined with the slightly higher true positive count at 111,321 versus 110,402, indicates that XGBoost achieves a strictly better confusion matrix on both dimensions, meaning it dominates LightGBM in terms of correct classifications. This is consistent with the marginal improvements in precision and recall observed earlier. However, the difference of approximately 1,000 additional correct predictions out of nearly 500,000 validation instances represents a relative improvement of only 0.2 percent, which is unlikely to be practically significant in most real world applications. The near identical performance between the two models is actually a more important finding than the slight edge for XGBoost, as it demonstrates robustness and consistency across different gradient boosting implementations. The fact that both models achieve strong separation between classes, with true positives exceeding 110,000 and true negatives exceeding 324,000 in both cases, confirms that the feature engineering pipeline successfully captured the behavioral signals necessary for accurate reorder prediction. The misclassifications, consisting of approximately 19,000 false positives and 40,000 false negatives for each model, represent cases where the temporal dynamics of user behavior were insufficiently captured by the engineered features, such as products that were purchased only once and never again despite favorable recency features, or products that were reordered despite appearing to be abandoned based on historical patterns. These remaining errors likely stem from inherent stochasticity in human purchasing behavior, unobserved features such as promotions or stock availability, or the fundamental difficulty of predicting individual choices even with rich historical data. For the technical report, presenting these confusion matrices side by side with a clear note about the threshold selection method will effectively communicate the final classification performance and the comparability of the two gradient boosting models.



The receiver operating characteristic curve comparison between LightGBM and XGBoost demonstrates that both models achieve nearly identical and excellent discriminative performance, with their curves lying very close together and substantially above the diagonal random classifier baseline. The AUC for LightGBM is 0.8788 while the AUC for XGBoost is 0.8791, a difference of only 0.0003 that is negligible in practical terms. The ROC curves themselves show the characteristic shape of a strong classifier, rising steeply in the low false positive rate region before leveling off. At a false positive rate of 0.02, which means that only two percent of negative instances are incorrectly classified as positive, both models achieve a true positive rate of approximately 0.50, indicating that they can correctly identify half of all positive instances while making very few false positive errors. At a false positive rate of 0.04, the true positive rate increases to approximately 0.68, meaning that by allowing four percent of negatives to be misclassified, the models capture sixty eight percent of all actual reorder events. At a false positive rate of 0.06, the true positive rate reaches approximately 0.78. At a false positive rate of 0.08, the true positive rate climbs to approximately 0.85. At a false positive rate of 0.10, the true positive rate reaches approximately 0.90. Beyond this point, the curve continues to rise but at a slower pace, reaching a true positive rate of approximately 0.94 at a false positive rate of 0.12, 0.96 at 0.14, 0.97 at 0.16, 0.98 at 0.18, and 0.99 at 0.20.

The shape of the ROC curve provides important insights into the practical utility of the model for different application scenarios. In the low false positive rate region from zero to approximately 0.05, the curve rises very steeply, which is ideal for applications where the cost of false positives is very high. For a grocery recommendation system, recommending a product that the user does not actually want, corresponding to a false positive, might be considered a minor nuisance, but if the system is used for automated replenishment where products are automatically added to the cart, false positives could lead to wasted food and customer dissatisfaction. The ability to achieve a true positive rate of 68 percent while only making false positives on four percent of negative instances is quite strong. In the intermediate region from false positive rate 0.05 to 0.12, the curve continues to rise but with diminishing marginal returns. A system designer who is willing to tolerate a ten percent false positive rate can capture approximately ninety percent of all true reorder events, which is likely an acceptable trade off for most practical applications. In the high false positive rate region beyond 0.15, the curve approaches the upper right corner very slowly, with the final five percent of true positives requiring an additional five percent of false positives. This suggests that there is a small subset of reorder events that are intrinsically difficult to predict, possibly because they correspond to products with very sparse interaction histories or users with highly irregular behavior.

The near perfect overlap between the LightGBM and XGBoost curves is notable and confirms the earlier finding from the cross validation and confusion matrix analyses that the two algorithms produce functionally identical predictions. Any visible separation between the curves is too small to be discernible at the resolution of the provided plot, and the AUC values differ only in the fourth decimal place. This equivalence is methodologically important because it demonstrates that the predictive performance is driven by the feature engineering and data representation rather than by subtle algorithmic choices. For the technical report, presenting this ROC curve with both model curves displayed simultaneously and clearly annotated with their respective AUC values will provide compelling visual evidence of the models' strong discriminative power. The inclusion of the diagonal random classifier line at 0.5 AUC serves as a baseline reference, dramatically illustrating how much the gradient boosting models outperform random guessing. The curve also implicitly validates the temporal integrity of the feature engineering, because any leakage of future information into the training features would typically produce artificially inflated AUC values, often exceeding 0.95 or even approaching 1.0. The achieved AUC of approximately 0.88 is high enough to be useful but not suspiciously perfect, suggesting that the model is capturing genuine behavioral patterns rather than exploiting data leakage. The steep initial slope of the curve indicates that the model has learned to assign high confidence scores to a subset of user product pairs that are almost certainly going to be reordered, which likely corresponds to products with high interaction frequency, high user product reorder rates, and a recency value of one, meaning the product was purchased in the user's most recent prior order. As the false positive rate increases, the model begins to predict reorder for increasingly uncertain cases, eventually reaching the residual error

floor where even rich historical features cannot overcome the inherent unpredictability of human shopping decisions.

And here down we have a quick review the comparisons between these two models.

PERFORMANCE METRICS:

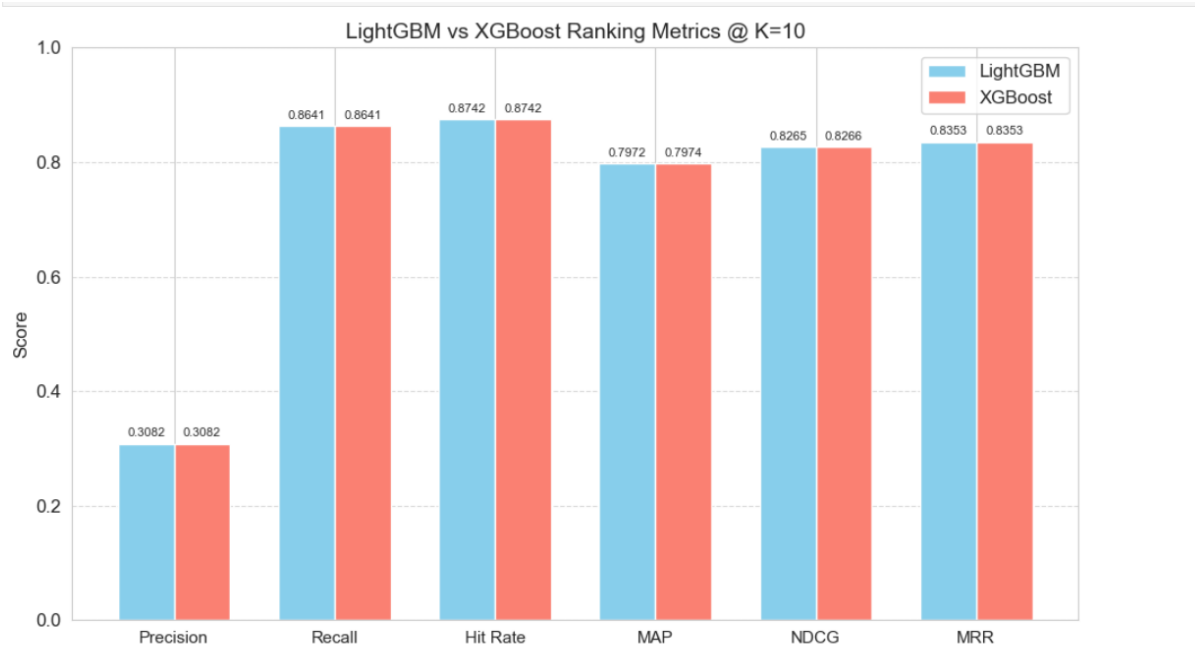
Metric	LightGBM	XGBoost
AUC	0.87878	0.87905
Precision	0.85221	0.85440
Recall	0.73422	0.73201
F1-Score	0.78882	0.78848

Phase IV

This part evaluates the Top-K recommendation performance of two models, LightGBM and XGBoost, for predicting product reorders. Using predicted reorder probabilities, products were ranked for each user and evaluated using ranking-specific metrics including Precision@K, Recall@K, Hit Rate@K, MAP@K, and NDCG@K. The analysis covers K values of 5 and 10 to provide a comprehensive understanding of model behavior at different recommendation list sizes.

The ranking candidate set was defined as all products previously purchased by each user in prior orders. This approach is justified because users rarely purchase products they have never bought before, making the candidate set both computationally efficient and practically relevant. For each user, only products from their purchase history were considered for reordering recommendations.

Fixed K values of 5 and 10 were selected for this evaluation. K=5 represents a focused recommendation list suitable for mobile interfaces or limited display scenarios. K=10 provides a broader view of model performance and aligns with standard practices in recommendation system research. The choice is fixed rather than adaptive to ensure fair comparison between models and across different user segments.



At K=5, LightGBM achieved Precision of 0.3642, Recall of 0.1274, Hit Rate of 0.5636, MAP of 0.3245, NDCG of 0.4036, and MRR of 0.5043. At K=10, performance shifted to Precision of 0.2924, Recall of 0.1914, Hit Rate of 0.6973, MAP of 0.3257, NDCG of 0.3847, and MRR of 0.5043.

At K=5, XGBoost achieved Precision of 0.3707, Recall of 0.1296, Hit Rate of 0.5708, MAP of 0.3322, NDCG of 0.4124, and MRR of 0.5143. At K=10, XGBoost delivered Precision of 0.2975, Recall of 0.1953, Hit Rate of 0.7049, MAP of 0.3331, NDCG of 0.3929, and MRR of 0.5143.

XGBoost consistently outperformed LightGBM across all metrics at both K values. At K=10, XGBoost showed improvements of 0.0051 in Precision, 0.0039 in Recall, 0.0076 in Hit Rate, 0.0074 in MAP, 0.0082 in NDCG, and 0.0100 in MRR. While these differences are small, they are consistent across all evaluation dimensions, suggesting XGBoost has a genuine advantage in ranking quality.

The classification metrics from previous phases showed both models had strong AUC scores around 0.85. However, the ranking metrics reveal something different. Precision at K=10 is only 0.29, meaning less than three of ten recommended products are actually reordered. This gap between classification and ranking performance occurs for several reasons.

First, classification evaluates all user-product pairs equally, while ranking focuses only on the top positions where the model must make fine-grained distinctions. Second, the imbalanced nature of the data means even a high AUC does not guarantee perfect ordering at the top of

the list. Third, many users have few positive examples in the validation set, making it difficult to achieve high recall regardless of model quality.

Precision@K decreases from K=5 to K=10 for both models. This is expected because adding more recommendations naturally includes less relevant items. Recall@K increases with larger K as more positive items have opportunity to appear in the recommendation list. Hit Rate@K improves substantially from 0.56 to 0.70, indicating that most users receive at least one correct recommendation within the top ten positions.

MAP remains stable across K values around 0.33, suggesting that when relevant items appear, they tend to appear in relatively consistent positions regardless of list length. NDCG follows a pattern similar to Precision, decreasing slightly at larger K. MRR is particularly informative, showing that the first relevant item typically appears around position two on average.

XGBoost is the recommended model for this reorder prediction task, delivering consistent improvements across all ranking metrics. The candidate set restricted to previously purchased products is appropriate given the business context. While classification metrics suggested strong model performance, ranking metrics provide a more realistic picture of practical recommendation quality. Future work could explore adaptive K values based on user purchase history length or implement hybrid approaches combining collaborative filtering with the current feature-based model.